Anti Bluff And Evidence

§ . NO BLUFF — positive-evidence-only validation

Every runtime test MUST satisfy ALL FIVE of the following non-negotiable constraints. A test that violates any one of them is a **bluff** and is forbidden from being committed to the test suite:

1. **Real ACTION**: the test MUST invoke at least one user-visible action via a project anti-bluff helper (e.g. `ab_send_action()` in shell-based suites). A test that records a PASS without ever calling such a helper is automatically failed by the summary function.
2. **State DELTA**: every PASS that claims to verify a feature MUST capture state BEFORE the action and AFTER, and assert the delta matches expectation. `before == after` on success is a bluff.
3. **POSITIVE EVIDENCE**: the PASS condition MUST be POSITIVE evidence (a value match, a state delta, a captured frame analysis, a captured audio frame analysis), NEVER absence-of-error. If a probe returns nothing, that's a FAIL not a SKIP.
4. **UNIQUE EVIDENCE TOKEN**: tests that interact with mutable framework state MUST embed a per-run UUID into the action AND look for that token in the resulting state. Cached results predating the token cannot match — this defeats the stale-cache false-pass.
5. **Audio / video features REQUIRE captured evidence**: features that produce audio output MUST be validated by capturing the actual output via a capture pipeline (loopback, hardware capture rig, or equivalent) — NOT by metadata-only checks. Features that produce video output MUST be validated via frame capture + OCR or pixel-difference analysis.

Each consuming project SHOULD ship a shared anti-bluff helper library (typically `tests/lib/anti_bluff.sh` or equivalent) that codifies these rules; every new runtime test MUST source it and use its assertion helpers.

8

§ . Bleeding-edge ultra-perfection quality bar

The acceptance bar for shipping a project change is **not** "tests pass and source compiles" — that is the floor. The acceptance bar is:

> **The user, on a freshly-deployed system, sees the expected effect when they perform the documented action — confirmed end-to-end, with captured evidence, on the actual target environment.**

The following are **forbidden**:

1. **"Source rebrand without shipped artifact rebuild"**: every source-side rename or rebranding MUST flow through to a shipped artifact that the runtime actually loads. Pre-build source-string gates are insufficient by themselves; shipped-artifact gates must accompany them.
2. **"Tests pass but feature broken"**: this is the failure mode §7.1 exists to eradicate. Any test that PASSes despite the feature being non-functional is a critical defect; its mutation pair must FAIL on the breakage.
3. **"Configuration-only tests"**: a test that only verifies a config file exists / has expected contents is downgraded to a pre-build gate. Runtime tests MUST exercise the actual user-visible behaviour and capture the result.
4. **"It works on my workstation"**: every fix is validated on every supported target before any release. No exceptions.
5. **"Will fix in next cycle"**: deferrals are documented in the changelog with concrete blockers + implementation plans, not just handwaved.

§ . Absolute codebase and data safety — zero risk, zero loss

Every destructive operation on the repository (history rewrite, force-push, branch deletion, bulk file removal, submodule de-init, pack repack that drops objects) is a **safety-critical** operation. Data loss from a wrong force-push is irreversible once the remote garbage-collects dangling objects. This section is non-negotiable.

§ . End-user quality guarantee — forensic anchor (User mandate, - -)

The reason §7.1 + §8 exist — captured verbatim from the project owner so that future engineers and AI agents understand the historical failure mode the covenant exists to eradicate:

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the canonical motivation for every gate, every mutation pair, every meta-test, every anti-bluff helper, every captured- evidence requirement, every multi-environment validation rule. The covenant exists because the project has historically shipped broken features behind PASS-ing tests, and that outcome is no longer tolerated.

Operative rule: the bar for shipping is not "tests pass" but **"users can use the feature."** Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Machine-created evidence at every gate (extension, 2026-08-15 — §11.4.262). Every claim by the System that something WORKS, PASSES, IS VERIFIED, or IS PRODUCTION-READY MUST cite MACHINE-CREATED, MACHINE-VERIFIABLE

evidence produced by the gate itself — not an operator's narrative, not "looks fine", not "no error was reported", not a prediction, not a guess, not an assumption. §11.4.262 makes this obligation EXPLICIT at every gate seam and applies §11.4.107(10) self-validating analyzers UNIVERSALLY, not only to AV playback. The captured-evidence family (§11.4.5 / §11.4.69 / §11.4.107) is REFINED from CAPTURED to CAPTURED-AND-MACHINE-VERIFIABLE. Prediction, guessing, and assumption about success are the exact anti-patterns this covenant forbids, at every layer of the System, from the first prompt of the session onward.

Propagation requirement: every `CLAUDE.md` and every `AGENTS.md` in the project tree (parent + every submodule recursively) MUST contain a clearly-titled "MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE" section that quotes this user mandate verbatim and points back to this §11.4 as the canonical authority. The consuming project enforces this via a pre-build gate (typically `CM-COVENANT-PROPAGATION`).

§ . . — FAIL-bluffs are equally forbidden

The covenant must be enforced symmetrically. The historical failure mode the covenant eradicates is the **PASS-bluff**: a green test result on a feature that doesn't work for users. But the inverse failure mode is just as toxic — the **FAIL-bluff**: a red test result caused by a script bug (shell crash, missing argument, regex error, malformed assertion) rather than by an actual product defect.

Examples of FAIL-bluffs:

- A test using `set -eu` calls `log_pass` without arguments → the helper references `$1` → script crashes with "1: parameter not set" → orchestrator records FAIL → engineer assumes product bug.
- A test pre-condition check uses an unbounded sed range → a comment line in the source matches the range terminator → the intended mutation never fires → audit reports PASS — bluff.

A FAIL produced by a test bug is no more useful than a PASS produced by a test bug — both mislead investigation, waste cycles, and ultimately let real defects ship undetected.

Operative rule (extended): every test MUST fail ONLY for genuine product defects. A test that crashes for a script-internal reason — undefined variable under `set -u`, unhandled error, malformed pipe, regex syntax error, division by zero, etc. — is a critical defect in the test suite itself and MUST be fixed at the source layer (helper library, shared lib, test source) so the failure mode disappears project-wide, not patched in individual call sites.

§ . . — Recorded-evidence requirement

A test that emits PASS without **captured visual or audio evidence of the user-visible feature actually working on the screen the user would see** is a §11.4 PASS-bluff. Closing this gap requires a project-wide recording + analyzer infrastructure consisting of (at minimum):

- **Recording wrapper** that captures the user-visible output to time-synchronized media files. Multi-display systems record every output stream in parallel; sync metadata (host wall-clock at spawn) is written alongside the recording.
- **Action-timeline emitter**: every test emits structured timeline events (JSONL or equivalent) at every user-visible transition.
- **Frame / audio analyzer**: scans the recordings, extracts samples at a configurable interval, feeds them to recognizers (OCR for text overlays, speech-to-text for audio, pixel-diff for frame health), correlates against the timeline, and emits findings.

Closure criterion: every PASS for a user-visible feature MUST be cross-checked by the analyzer against the recording + action timeline. A PASS that lacks at least one matched timeline event in the analyzer findings is treated as a §11.4 PASS-bluff.

§ . . — Per-environment-topology test dispatch

Tests that depend on environment topology (presence/absence of secondary displays, microphones, network interfaces, peripheral devices, cloud services, etc.) MUST detect topology at test entry and dispatch the topology-appropriate variant. A

test that runs the WRONG variant for the actual topology and PASSes is a §11.4 PASS-bluff: it claims a feature works while never having exercised it on the configuration the user actually has.

A topology-touching test that does NOT have a dispatcher AND a per- topology variant is a §11.4 violation regardless of how it "passes" on any specific configuration. Single-test-multi-topology scripts MUST gate every topology-dependent assertion behind an explicit topology check that emits SKIP-with-reason if the required topology is absent — never PASS-by-default.

§ . . — Test-interrupt-on-discovery + retest-from-clean-baseline

A test cycle that *continues running past a freshly discovered defect* is itself a §11.4 PASS-bluff: it produces "all green" summaries that the operator might tag as a release while the codebase under test is known-broken at the moment those greens were recorded.

The mandatory protocol — non-negotiable:

1. **Interrupt immediately.** The moment a defect is re-discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop. No "let it finish for the data" — the data is contaminated.
2. **Systematic debugging before any fix.** Identify root cause at source layer (not symptom), blast radius across related tests/features/subsystems, and regression-protection seam. Symptom patching is forbidden.
3. **Fix at root cause** per §11.4.1: source-layer fix, not symptom patch in the calling site. The fix MUST be safe (no new failure modes), minimal (no surrounding cleanup smuggled in), and non-regressive.
4. **Four-layer test coverage per fix.** Every fix lands with positive-evidence coverage in every applicable layer:
    - **Pre-build / pre-merge gate** (source-level catch)
    - **Post-build / packaging gate** (artifact-level catch)
    - **Runtime / on-device test** (end-user-behaviour catch)
    - **Meta-test paired mutation** (proves the gate is not itself a bluff)

5. **Documentation MUST be updated for every fix** — Issues → Fixed migration on closure, Applied Fixes table row, user-facing guides, architecture diagrams, per-version changelog.
6. **Rebuild the system.** Full build via the canonical build script. Skipping rebuild — even when the fix is host-only — invites stale-artifact contamination of the retest baseline.
7. **Re-deploy on every supported target.** A target left on the pre-fix build is a confounder.
8. **Repeat the full test suite** from the beginning on every target, sequentially per host memory cap. Partial reruns are a §11.4 violation.
9. **No-bluff certification per cycle.** Before tagging: meta-test harness returns all gates green AND every gate's paired mutation FAILs.

§ . . — Captured-evidence quality analysis

§11.4.2 mandates *captured* evidence; §11.4.5 mandates the **content** of that evidence be analyzed for quality, not merely for presence.

Audio quality analysis — every audio test that PASSes MUST verify:

1. **Presence** — non-trivial RMS amplitude in captured output.
2. **Channel count** — analyzer reports the exact channel count the test claims (2.0 stereo, 5.1 surround, etc.). Stereo downmix in a "5.1 PASS" claim is a PASS-bluff.
3. **Sample rate + bit depth** — match the codec / pipeline under test.
4. **Glitch census** — underruns, XRUNs, dropouts above tolerance MUST be explicitly classified (PASS within budget, WARN above, FAIL on hard limits). Silently ignoring counts is a PASS-bluff.
5. **Coexistence-artifact census** — radio / IPC / shared-resource contention above tolerance.

Video quality analysis — every video test that PASSes MUST verify:

1. **Presence** — captured recording has non-zero size AND decoded- frame total > 0. A 0-byte file is the canonical PASS-bluff.

2. **Routing target** — analyzer confirms video appeared on the intended display / surface.
3. **Frame health** — drop count, jitter, freeze detection, tearing.
4. **Obstruction census** — OCR scan for hostile overlays (error dialogs, sign-in dialogs, geo-restriction overlays, paywall, etc).
5. **Resolution + codec** — captured dimensions match what the test claims.

§ . . — No-guessing mandate

Tests, gates, status reports, closure narratives, commit messages, and any operator-facing text MUST NOT use words like `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, `appears to`, `guess`, `seemingly`, `apparently`, `perhaps`, `supposedly`, `conjectured`, or their synonyms when describing CAUSES of test failures, system behaviour, or fix effectiveness.

Either:

1. **Prove the cause** with captured forensic evidence (logs, kernel ring buffer, kernel ramoops, structured snapshots, OS journals, strace, etc.) and state it as fact, OR
2. **Explicitly mark UNCONFIRMED + PENDING_FORENSICS** with a tracked-task ID for follow-up.

Every "X likely caused Y" sentence in the codebase or documentation is a §11.4.6 violation. Every "appears to be benign" without a concrete forensic trace is a §11.4.6 violation.

§ . . — Demotion-evidence rule

A demotion from any FAIL classification (`OPEN`, `POSSIBLE PRODUCT DEFECT`, `FAIL`) to a lower-severity classification (`INVESTIGATED`, `MITIGATED`, `RESOLVED`, `WORKING-AS-INTENDED`) requires positive evidence captured under the SAME CONDITIONS that originally exposed the defect:

1. **Same target** — defect on Target-A needs Target-A evidence; cross-target claims need every target.

2. **Same build** — evidence captured BEFORE a relevant fix landed does not validate post-fix behaviour.
3. **Same cycle position** — a stress-soak FAIL is not refuted by an isolated-test PASS.
4. **Same load profile** — a contention-mode FAIL is not refuted by a quiescent-mode PASS.

"I cannot reproduce in isolation" is a HYPOTHESIS, not a finding. Per §11.4.6 it MUST be tagged `UNCONFIRMED:` until same-conditions retest produces positive evidence.

§ . . — Out-of-band sink-side captured-evidence

Whenever a downstream consumer (HDMI sink, cloud service, monitoring target, downstream system) provides a network-accessible introspection API that reports what was actually received, the test suite MUST consume that report as captured-evidence for every test asserting end-to-end delivery.

The on-source-side view ALONE is insufficient — that is the exact "tests pass but the feature doesn't work" pattern §11.4 forbids. Sink-side reports MUST be:

- Identity-verified (MAC, serial, UUID match) before consumed as evidence.
- Topology-dispatched (§11.4.3) — sink-probe-unreachable → SKIP, never FAIL.
- Cross-referenced with the on-source state at a matching wall-clock instant.

§ . . — Installable-Asset Evidence Mandate (User mandate, - -)

Forensic anchor — verbatim operator report (2026-05-17):

> "app does not have a launcher icon anymore — the latest published app tester build(s). how come app passed anti-bluff checks?"

For any user-distributable build artifact (package, bundle, installer, or container image produced by the build pipeline and distributed to end users), tests and challenges MUST open the artifact and verify each user-visible asset is **present** and **non-degenerate**.

"User-visible asset" includes (non-exhaustive):

- Application icons for every density / resolution tier declared in the artifact metadata, including any platform-specific icon format (multi-resolution container, adaptive XML, symbol set, etc.) that the target OS uses when the minimum supported OS version requires it.
- Splash screens declared in the install metadata.
- Application name strings as declared in the install metadata.
- Any other asset whose absence causes the user to be unable to identify, launch, or interact with the installed application from the OS launcher / home screen / app drawer.

A PASS without opening the artifact and verifying the asset chain end-to-end is a §11.4 PASS-bluff, regardless of whether source files exist and tests otherwise pass. The specific failure mode this rule targets is: source file exists → build pipeline packages it → post-build checks pass at the source layer → artifact ACTUALLY produced with the asset stripped or misconfigured, and no gate ever opens the artifact to verify.

Required evidence: the anti-bluff challenge for a user-distributable artifact MUST produce per-asset PASS/FAIL lines showing: (a) the asset entry exists in the artifact package listing, (b) the asset is non-empty / non-degenerate (size ≥ platform-defined minimum OR format-validated), (c) where the OS's asset-resolution path involves indirection (alias, XML reference chain, density-qualifier override), the full chain is traced to the final rendered resource.

Consuming-project implementation: each consuming project ships one challenge script per artifact type that opens the produced artifact and verifies every declared user-visible asset. The challenge MUST run as part of the project's standard QA gate (equivalent of `make qa-all`).

Root cause of §11.4.38 introduction: a consuming project shipped multiple releases with the application icon absent on the target OS because: (1) the icon XML used a resource type that the OS resolves differently above a specific API

level, (2) all pre-ship challenges verified source files only, none opened the packaged artifact. This is a pure §11.4 PASS-bluff — every test and challenge reported green while the end user saw no icon.

Classification: universal (per §11.4.17). No escape hatch. Severity- equivalent to a §11.4 PASS-bluff at the artifact-packaging layer. Composes with §11.4.1–§11.4.5 (evidence requirements), §11.4.25 (full automation coverage), §11.4.27 (no fakes beyond unit tests — source-layer checks of a distributable artifact are the distributable- layer analog of unit-test-only coverage). See Constitution §11.4.38 for the full mandate.

§ . . — Positive sink-side / downstream evidence mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!"

§11.4.13 already mandates consuming the Arvus sink-side report when available. §11.4.68 extends this with a stricter contract: **a test that asserts audio or video routing PASS MUST capture and verify positive sink-side or downstream evidence — never config-only, never metadata-only, never PCM-open-state-only.** The §11.4.13 path described what SHOULD be captured; §11.4.68 fixes the failure mode where tests "PASSed" because the helpers silently SKIPped the very evidence the assertion depended on.

Positive sink-side / downstream evidence — closed enumeration (at least one MUST be captured for every audio/video routing PASS; none alone is sufficient if the canonical sink/downstream is reachable):

1. **Sink-side codec-state** (Arvus / Sonos / Yamaha / Denon / Marantz / WiSA REST API) showing a **non-empty Codec-In-Use** value matching the test's expected codec regex during the playback window. Empty / `<unreachable>` / `<N.E.>` / `<None>` placeholders are NOT positive evidence.
2. **PCM frames-written delta** from `/proc/asound/cardN/pcmMp/sub0/ status hw_ptr` — the delta over the playback window MUST be strictly positive AND consistent with the expected sample-rate × channel-count × duration. Zero or negative delta is FAIL.
3. **ALSA ELD / EDID-Like-Data** from `/proc/asound/cardN/eld#X.Y` demonstrating the sink negotiated the expected channel count + format. Empty ELD is NOT evidence the sink received audio — it only proves the cable / EDID-channel is up.
4. **ffprobe-on-captured-mp4** for video routing — non-zero frame count, expected codec, expected resolution, expected fps. 0-byte capture (Bug #24 pattern) is FAIL.
5. **Recording-analyzer event match** per §11.4.2 / §11.4.5 timeline — at least one matched event in the analyzer findings.
6. **Tinycap RMS amplitude** for ALSA-loopback audio recordings — non-trivial RMS in the captured WAV (>= -60 dBFS for line-level playback).

Failure-mode taxonomy — what §11.4.68 specifically forbids:

- `arvus_probe_present` returns 1 → test reports SKIP → suite reports all-green. **FORBIDDEN.** When the sink-side evidence is REQUIRED by the assertion, missing sink is `OPERATOR-BLOCKED` (an explicit release-blocker, NOT a SKIP, NOT a PASS).
- `arvus_probe_codec_state` returns empty string → test logs "codec_state: " (blank) → assertion proceeds on HAL-side metadata. **FORBIDDEN.** Empty codec-state IS a defect signal — either the audio is not arriving at the sink (genuine product defect per current User mandate) OR the sink is unreachable (operator-blocked). Either way, the test cannot PASS.

- Test reads HAL `dumpsys media.audio_flinger` output-thread device field showing `0x400 (AUDIO_DEVICE_OUT_HDMI)` → reports "HDMI routing PASS". **FORBIDDEN.** That field reflects what the audio POLICY chose, NOT what the audio HAL physically opened. The HDMI ALSA card may still be closed (PCM open failure swallowed by the HAL's `ALOGW` fallback) — silent silence at the user's ear. Sink-side codec-state or PCM `hw_ptr` delta is the ONLY proof audio actually arrived.
- Test asserts `/vendor/etc/audio_policy_configuration.xml` contains the `deep_buffer` route to `HDMI Out`. **FORBIDDEN as the only evidence.** Config-only PASS is the canonical §11.4 PASS-bluff — config can be perfect AND audio still inaudible (e.g., the HAL may still misroute). MUST be paired with at least one runtime positive evidence from the enumeration above.

Mandatory protections (all four):

1. **Library contract.** Sink-side helper libraries (`arvus_probe.sh` , `sonos_probe.sh` , etc.) MUST expose a `*_require_reachable` function that returns exit code 2 (OPERATOR-BLOCKED) when the sink cannot be probed. Tests asserting sink-side codec-state MUST call this function at probe entry. Silent skip via `*_probe_present` is permitted ONLY for topology-dispatch fan-out logic that already has an alternative on-SoC + downstream-amplitude probe path.
2. **Exit-code propagation.** `arvus_require_reachable` / equivalent helpers return 2; the test harness MUST propagate 2 to the suite summary (counted separately from FAIL/SKIP) as `OPERATOR-BLOCKED` . `test_all_fixes.sh` orchestrator MUST exit non-zero when any test reported `OPERATOR-BLOCKED` AND release tags MUST NOT be cut while any `OPERATOR-BLOCKED` test exists.
3. **Test rewrite.** Every test currently asserting `audio output routing PASS` via metadata-only checks (e.g., the historical `test_audio_output_routing.sh` config-only invariants) MUST be rewritten to capture at least one positive sink/downstream evidence per the enumeration above. The §11.4 PASS-bluff pattern `tinymix says route is on → PASS` is the failure mode this anchor closes.
4. **Anti-stickiness post-stop.** After the test stops the playback, it MUST re-probe the sink/downstream and assert the codec-state transitioned to N.E. / N/A / 0-frames-delta. A persistent codec- state across test boundaries indicates orphan playback (§11.4.14 violation) AND would falsely PASS the next test.

Pre-build gate `CM-COVENANT-114-68-PROPAGATION` enforces this anchor literal across canonical files + per-consumer propagation. Paired mutation strips the anchor literal → gate FAILs. Pre-build gate `CM-POSITIVE-SINK-EVIDENCE-PER-AUDIO-TEST` walks every audio-routing on-device test asserting PASS and verifies it cites at least one positive evidence call (one of `arvus_require_reachable` / `arvus_assert_codec_format` / `pcm_hw_ptr_delta_positive` / `tinycap_rms_above_floor` / `ffprobe_frames_nonzero`). Paired mutation strips one such call → gate FAILs.

Composes with §11.4.2 (recorded-evidence — sink-side is the captured-evidence channel), §11.4.5 (audio + video quality analysis comprehensiveness — codec / channel-count / RMS), §11.4.13 (sink-side captured-evidence mandate — §11.4.68 closes its silent-skip gap), §11.4.14 (test cleanup — anti-stickiness post-stop), §11.4.46 (recent-work validation — sink-side evidence required at validation entry), §11.4.49 (dual-approach testing — both UI + Intent variants need sink-side evidence), §11.4.50 (deterministic consistency — sink-side evidence must be consistent across N iterations), §11.4.52 (autonomous-validation — sink probe IS the canonical autonomous path for HDMI audio).

No escape hatch — no `--skip-sink-evidence`, `--allow-empty-codec`, `--sink-unreachable-is-pass`, `--metadata-only-suffices` flag exists anywhere. The discipline exists because the operator/end-user forensic on 2026-05-20 demonstrated the exact failure this anchor forbids: tests reported audio-routing PASS while the user heard nothing and Arvus Codec-In-Use was empty.

Canonical authority: constitution submodule `Constitution.md` §11.4.68.

Non-compliance is a release blocker regardless of context.

2026 05 20

§ . . — Universal Sink-Side Positive-Evidence Taxonomy + Mechanical Enforcement (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! Dive deep into the root causes and how this had happened! The root causes of such omission MUST BE TRACKED, and proper solution applied! Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! Make sure everything is added we change as mandatory rules we must follow and critical constraints into ours root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md!!! Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment! THIS IS MOST CRITICAL POINT WE HAVE NOW with all audio issues!"

Escalation from the same operator session (2026-05-20, hours earlier):

> "We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! ... How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working!"

Forensic incident — 2026-05-19→20 D3 audio-routing PASS-bluff generalised. §11.4.68 closed the audio-specific failure path where Arvus reported an empty Codec-In-Use field and the test PASSed anyway. The same shape of bug had previously shown up intermittently across multiple subsystems (display routing on D1 secondary HDMI, BT A2DP codec advertisements, WiFi connection quality, video playback frame-count, touch-input event delivery). The audio incident was the canonical trigger because the operator hears silence at the soundbar even while the cycle reports green. §11.4.69 is the universal generalisation — closing the bluff class for every user-visible feature, not only audio.

Root-cause analysis — why existing anchors §11.4.2 / §11.4.5 / §11.4.13 / §11.4.27 / §11.4.52 / §11.4.68 did NOT mechanically catch the failure across the System:

1. **Aspirational text without a pre-build gate.** §11.4.2 (recorded-evidence) required captured visual/audio evidence per PASS but had no gate walking the test corpus to enforce a per-PASS evidence file. Tests called `ab_pass "description"` and the evidence requirement was honour-system.
2. **SKIP-fail-open hiding missing evidence.** §11.4.13 (Arvus sink-side) had a silent-skip escape when the sink returned an empty / unreachable response — the empty response was the very defect signal but it was classified as "sink unreachable, SKIP the assertion." §11.4.68 closed this for audio; §11.4.69 closes it universally across every feature class with a downstream sink.
3. **PASS-by-default vs FAIL-by-default helper contract.** The bare `ab_pass` helper accepted a free-text description and incremented the PASS counter — no evidence path required, no shape enforced. The default failure mode was therefore PASS- bluff. The correct default is FAIL-by-default — a PASS must prove itself with a captured-evidence artefact path.
4. **Missing closed-set taxonomy.** Individual test authors made one-off decisions about what evidence shape was acceptable. Without a single canonical table mapping every user-visible feature class to its required sink-side probe, aggregate enforcement across the System was mechanically impossible.
5. **Helper functions that mapped a multi-step probe to one bool.** `arvus_probe_present` collapsed reachable-but-empty into the same value as unreachable. The §11.4.68 fix added `arvus_require_reachable` for audio; §11.4.69 codifies the equivalent for every sink (Sonos, Yamaha, Denon, video display analyzer, WiFi sink, BT peer, etc.).
6. **No per-feature evidence ledger.** §11.4.52 supplied an autonomous-validation classification (`AUTONOMOUS_VERIFIED` etc.) but did not require each test to cite a captured-evidence artefact path matching the §11.4.69 taxonomy.

The §11.4.69 mandate — universal, generic, applies to every System component, every owned project, every consumer subscribing via the constitution submodule:

Element 1: Closed-set sink-side / downstream evidence taxonomy. Every user-visible feature class in every consuming project MUST map to exactly one entry in the following table. The taxonomy is the canonical authority on the required evidence shape per feature class. Tests annotate themselves with `# §11.4.69 FEATURE: <class>` so the pre-build gate can match expected evidence shape.

Feature class	Required probe	Required evidence shape
`audio_output`	sink-side codec/channel REST probe (Arvus / Sonos / Yamaha / Denon / Marantz / WiSA) + on-device `tinycap` capture + ffprobe channel assertion	non-empty `codec_in_use` matching expected codec AND captured WAV with RMS > -60 dBFS AND ffprobe `channels` matches claim
`audio_input`	tinycap capture from intended device + ffprobe RMS	captured WAV with RMS > -60 dBFS AND `audio_devices_for_attr` shows the claimed source device
`video_display`	`screenrecord` on intended display + `ffprobe -count_frames` + analyzer event-match	non-zero mp4 size AND `nb_read_frames > 0` AND analyzer reports a matched event for the claimed display
`network_throughput`	`iperf3` / `curl --speed` / `dd` over network	throughput ≥ per-link-type floor AND TCP buffer / congestion-control matches claim
`network_connectivity`	ping + DNS resolve + TCP connect probe	captured ping reply AND DNS answer AND TCP handshake to known port
`bluetooth_a2dp`	codec query + sink peer state + A2DP TX queue depth	`dumpsys bluetooth_manager` reports `CONNECTED+A2DP_PLAYING` AND codec field non-empty AND queue depth < overflow threshold
`bluetooth_pair`	`dumpsys bluetooth_manager` + `BluetoothDevice.getBondState()`	bond state = `BOND_BONDED` AND device MAC matches expected

Feature class	Required probe	Required evidence shape
`touch_input`	`getevent -lt` capture during scripted `input tap`	≥1 `EV_ABS ABS_MT_*` event per scripted tap AND `dumpsys input` shows the touch device active
`sensor`	`dumpsys sensorservice` + `getevent` on sensor device	event count > 0 between BEFORE and AFTER snapshots for the claimed sensor
`gpu_render`	SurfaceFlinger frame-rate dump + `dumpsys gfxinfo <pkg>`	`Janky frames` < threshold AND `Total frames rendered > 0` during test window
`storage_read`	`dd if=<path> of=/dev/null` + `/proc/diskstats` delta	throughput ≥ floor AND `read_sectors` delta confirms IO landed on intended device
`storage_write`	`dd of=<path>` + sync + `/proc/diskstats` delta	throughput ≥ floor AND `write_sectors` delta confirms IO landed on intended device
`mediaco-dec_decode`	`media.metrics` dump + `dumpsys media.codec`	decoder lifecycle events present (`configure` , `start` , ≥1 output buffer) AND codec name matches claim
`mediaco-dec_encode`	`media.metrics` dump + non-zero output file	encoder lifecycle events present AND output file non-zero AND ffprobe confirms codec matches claim
`miracast` / `cast`	sink-side WFD / Cast announce + session state	sink dashboard reports active session AND session ID present in `dumpsys media_session`

Feature class	Required probe	Required evidence shape
`boot_service`	`getprop init.svc.<name>` + log capture	service state = `running` AND log shows expected boot-time output literal
`package_install`	`pm path <pkg>` + signature match	non-empty pm path AND APK signature matches expected platform key
`permission_grant`	`dumpsys package <pkg>` permission state	permission state = `granted: true` for the claimed permission
`wifi_link`	`iw dev wlan0 link` + RSSI + frequency	captured BSSID AND RSSI within range AND frequency matches claim
`wifi_throughput`	`iperf3` to LAN server	throughput ≥ floor for the negotiated PHY rate
`ethernet_link`	`ip link show` + carrier state + throughput probe	`carrier=1` AND `speed` matches claim AND throughput probe succeeds
`display_topology`	DRM connector dump + display ID enumeration	connector status `connected` for claimed connector AND display ID present in `dumpsys SurfaceFlinger`
`drm_playback`	Widevine session + decoded-frame count	active DRM session AND decoded-frame count > 0
`subtitle_render`	analyzer + Tesseract OCR on captured frames	OCR matches expected subtitle text on the claimed display

The taxonomy is **open to additions** via constitution-submodule commits + propagation. Removal of a feature class is forbidden without a tracked work item explaining the supersession. Consumer projects MAY extend the taxonomy with project-specific feature classes (e.g., a consuming project's build-pipeline classes) but MUST NOT contract it.

Element 2: New helper contracts (additive during grace period; mandatory after grace-period end-date 2026-06-19).

The project anti-bluff helper library (`anti_bluff.sh` in a consuming project; functionally equivalent helper in every consuming project) MUST provide three new helpers:

1. **`ab_pass_with_evidence <description> <evidence_path>`** — the new canonical PASS helper:
   - REQUIRES a non-empty `evidence_path` argument
   - Verifies the path exists AND is non-empty (`[ -s "$path" ]`)
   - If the path is missing or empty, the helper EXITs the test as FAILed with a diagnostic citing the missing evidence path
   - On success, increments the PASS counter AND emits a result line `PASS: <description> [evidence: <path>]`
   - Composes with `ab_send_action` — a single test issues many `ab_send_action` calls and one terminal `ab_pass_with_evidence`
2. **`ab_skip_with_reason <description> <reason_code>`** — the new canonical SKIP helper:
   - REQUIRES `reason_code` from a CLOSED set: `geo_restricted` , `operator_attended` , `hardware_not_present` , `topology_unsupported` , `network_unreachable_external` , `feature_disabled_by_config` , `artifact_not_yet_built`
   - REJECTs ad-hoc reasons — any string not in the closed set fails the helper at call time
   - `artifact_not_yet_built` (added 2026-07-17, research-derived) names the NOT-YET-RUNNABLE class: the check is CORRECT and the topology IS present, but its precondition artifact does not exist yet (the fix is not built / not deployed to the target). It is the legal string for the §11.4.135 REQUIRES-REBUILD PENDING class, which previously had NO code in

this closed set and therefore had to either misuse a false code (`hardware_not_present` / `feature_disabled_by_config` — both untrue, and a §11.4.6 misstatement of the real reason) or fail the helper at call time. Reporting a not-yet-runnable check as FAIL is a §11.4.1 FAIL-bluff (it sends teams to fix healthy code); reporting it as PASS is a §11.4 PASS-bluff. Its seam-dependence is §11.4.135's, NOT flat: on an intermediate artifact it is honest and non-blocking; on the release candidate it is a contradiction that BLOCKS (absence of a verdict blocks exactly as a FAIL does). A flat "always blocks" rule is REJECTED per §11.4.135 as itself a §11.4.201 FAIL-bluff. Consumer-migration note (§11.4.164 seam): adding a member to a CLOSED set means every existing `ab_skip_with_reason` implementation REJECTS the new code at call time until updated — update the helper alongside the constitution pull.
    - FORBIDs `network_unreachable_external` for any feature class in the §11.4.69 taxonomy that lists a sink-side probe — that escape was the §11.4.13 fail-open path §11.4.68 closed for audio, and §11.4.69 closes universally
    - SKIP remains mechanically distinct from PASS in cycle math (per §11.4.1)
3. **Bare `ab_pass` deprecation wrapper.** The legacy `ab_pass` helper MUST emit a WARNING line on every call AND emit a deprecation event to a tracked log file (`/data/local/tmp/ab_pass_deprecations.log`). After the grace period (defined in Element 4) the wrapper MUST FAIL the test outright, forcing all callers to migrate.

Element 3: Mechanical enforcement via three pre-build gates + three paired meta-test mutations (per §1.1).

1. **`CM-SINK-EVIDENCE-PER-FEATURE`** — walks every `.sh` test in `device/rockchip/rk3588/tests/` (and equivalent project paths), parses for `# §11.4.69 FEATURE: <class>` annotation comments, verifies the test invokes the corresponding sink-side probe from the taxonomy AND uses `ab_pass_with_evidence` (or `ab_skip_with_reason` with a permitted reason). Pre-grace: tests lacking the annotation WARN. Post-grace: they FAIL.
2. **`CM-NO-FAIL-OPEN-SKIP`** — audits sink-side probe helpers (`arvus_probe.sh` , `sonos_probe.sh` , equivalents) for any code path that converts an empty / unreachable sink response into a PASS-counting SKIP for a feature class with a

sink-side probe in the taxonomy. The gate FAILs if any such path exists. Remediation: escalate to FAIL (the empty response IS the defect) or to OPERATOR-BLOCKED per §11.4.68.

3. **`CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE`** — pre-grace WARNs on every bare `ab_pass` call in in-scope tests; post-grace (2026-06-19) FAILs the gate. Tests that legitimately have no evidence-capable PASS path MUST migrate to `ab_skip_with_reason operator_attended` and add a tracked work item per §11.4.52's `OPERATOR_ATTENDED_ONLY` classification.

Paired mutations (per §1.1):

- **Mutation 1:** strip the literal `ab_pass_with_evidence` from the project anti-bluff library → assert `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` FAILs.
- **Mutation 2:** inject a fresh test that calls bare `ab_pass` without the `# §11.4.69 FEATURE:` annotation → assert `CM-SINK-EVIDENCE-PER-FEATURE` FAILs post-grace, WARNs pre-grace.
- **Mutation 3:** inject a fresh test that calls `ab_skip` (no `_with_reason` suffix) OR `ab_skip_with_reason network_unreachable_external` for an `audio_output` feature → assert `CM-NO-FAIL-OPEN-SKIP` FAILs.

Element 4: Grace period mechanics — additive, not destructive.

- **Grace period:** 30 days from anchor land date — 2026-05-20 through 2026-06-19 inclusive. The end date is hardcoded in the helper library and in this anchor; no environment variable, no command-line flag, no operator override can extend or shorten it.
- **Pre-grace behaviour:** new helpers exist and are usable; the three gates emit WARN for legacy patterns; cycle math counts WARNs separately from PASS / FAIL / SKIP.
- **Post-grace behaviour:** WARN promotion to FAIL is automatic via a date check (`[ "$(date -u +%Y%m%d)" -ge 20260619 ]`) inside each gate's enforcement logic.

Element 5: Inheritance via HelixConstitution. This anchor lives in the constitution submodule and applies to every consuming project that subscribes — including Catalogizer and every future consumer. Each consumer MUST implement its own

equivalent helper library + pre-build gates; the taxonomy in Element 1 is the canonical authority across all consumers. Consumer-specific extensions are allowed only as additions to the taxonomy.

Composes with:

- §11.4.1 (FAIL-bluffs equally forbidden — `ab_skip_with_reason` closed-set prevents script-bug FAILs hiding real defects)
- §11.4.2 (recorded-evidence requirement — Element 2 helper contracts make captured evidence mechanically required per PASS)
- §11.4.5 (audio + video quality 5-layer — §11.4.69 enforces that the layers are exercised before PASS via the taxonomy probe)
- §11.4.6 (no-guessing — sink-side evidence eliminates "we think audio is routing" guessing; the codec field reads concretely)
- §11.4.13 (Arvus sink-side evidence — §11.4.69 mechanically closes the SKIP-fail-open escape §11.4.13 left open)
- §11.4.27 (no-fakes-beyond-unit — §11.4.69 adds the runtime hook §11.4.27 lacked)
- §11.4.50 (deterministic consistency — sink-side evidence is what `ab_run_n_times` hashes for divergence detection)
- §11.4.52 (autonomous-validation — §11.4.69 supplies the evidence-shape contract per feature class; §11.4.52 supplies the classification ledger)
- §11.4.68 (positive sink-side / downstream evidence for audio + video — §11.4.69 is the universal generalisation across all feature classes)

No escape hatch:

- No `--skip-evidence` flag.
- No `--config-only-pass` flag (configuration-only PASS is the exact bluff pattern this anchor closes).
- No `--allow-fail-open-skip` flag.
- No `--legacy-ab-pass-permitted` flag post-grace.
- No taxonomy-bypass shortcut — every feature class in scope MUST have its evidence shape enforced.

Propagation gate. `CM-COVENANT-114-69-PROPAGATION` verifies the §11.4.69 anchor literal is present across the ~44-file consumer fleet (parent CLAUDE.md / AGENTS.md + Containers + 10 owned atmosphere submodules including the smarttube-player nested set + 7 HelixQA submodules). Constitution submodule files (constitution/{Constitution,CLAUDE,AGENTS}.md) are canonical authority — they carry §11.4.69 by definition.

Canonical authority: constitution submodule `Constitution.md` §11.4.69.

Non-compliance is a release blocker regardless of context.

§ . . — docs/qa/ end-user evidence mandate (User mandate, - -)

The mandate. Every feature that ships MUST carry a recorded end-to-end communication transcript plus any attached materials (screenshots, request/response payloads, audio, file uploads) committed under `docs/qa/<run-id>/` — one directory per feature run. A feature with no QA transcript is itself a §11.4 / §107 PASS-bluff: it claims to work but carries no auditable runtime evidence that an end user actually exercised it through the same interface they will use in production.

Forensic anchor (verbatim user mandate, 2026-05-22):

> "every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof."

Operative rule.

1. Every consuming project MUST maintain a `docs/qa/` tree (no exception, no escape hatch). Each new feature run lands under `docs/qa/<run-id>/` where `<run-id>` is monotonic + greppable (timestamp, HRD-NNN, ATM-NNN, or other workable-item identifier — see §11.4.54).
2. The transcript MUST be full bidirectional — every prompt/command sent + every response received + every error message + every state change observed. One-sided ("we sent X") is not a transcript; both halves are required.

3. Attached materials MUST be committed alongside (screenshots in `.png`, payloads in `.json` / `.toon` / `.txt`, audio in `.wav` / `.mp3`, etc.). External-only links (Slack URL, Drive URL) are §11.4.13 sink-side violations — the evidence MUST live in-repo.
4. Bot-driven / agent-driven automation (a consuming project's QA bot, e.g. Herald's planned `qaherald`, or analogous binaries in other projects) MUST preserve the full conversation thread as the proof artefact. A bot that runs the round-trip but stores only the final PASS/FAIL line is itself a §107 bluff at the QA-automation layer.
5. CI / release gates MUST refuse to tag a version that has any feature-shipping commit without its matching `docs/qa/<run-id>/` directory present and non-empty. The `release` script in the consuming project enforces this; the constitution submodule's `scripts/qa_coverage_audit.sh` (when implemented) gives the universal scanner.

Composes with §11.4.2 (recorded-evidence requirement — `docs/qa/` is the canonical capture location for end-user-facing features), §11.4.5 (captured-evidence quality analysis applies per-transcript), §11.4.13 (sink-side evidence stays in-repo, not behind external links), §11.4.65 (multi-format export — transcripts may be exported to PDF/HTML siblings when operator-readable), §11.4.69 (universal sink-side positive-evidence taxonomy — `docs/qa/` is the "user-facing-channel" branch), §107 (an end-user feature without an end-user-channel transcript is a §107 PASS-bluff), §1.1 (paired mutation: delete a `docs/qa/<run-id>/` → release gate FAILs).

Classification: universal (per §11.4.17). Every project under this Constitution carries this rule. Single-feature pre-implementation projects are exempt until their first user-visible feature lands.

Canonical authority: constitution submodule `Constitution.md` §11.4.83.

Non-compliance is a release blocker. No `--qa-evidence-optional`, `--qa-transcript-later`, `--qa-bot-summary-suffices` flag exists.

---

§ . . — Natural-language intent recognition & clarification (User mandate, - -)

Short tag: `intent-recognition-clarification`.

Forensic anchor — verbatim user mandate (2026-05-31):

> "Users must NOT need to know command syntax (no `COMMAND: …` prefix). They send a clear natural-language message; the System determines the intent. The System recognizes the commands it has; if none matches it infers the exact intent; if it is totally unable it replies, tags the user (`@user …`), and asks to clarify precisely. We MUST always do our best to determine exact intent so we never annoy end users. This is a CORE part of the System."

This anchor binds natural-language intent recognition as a MANDATORY constraint on every consumer that ships a messenger/notification-driven command surface (Herald and its flavor binaries are the reference implementation; a consuming project and any future messenger-bearing project inherit per §11.4.35). The detailed normative spec is the Herald design contract `docs/design/INTENT_RECOGNITION.md` (three-tier resolution, command set, the `clarify` action, envelope wiring) — this section restates its load-bearing constraints; implementations code against the contract, not against a paraphrase.

(A) No required command syntax. Users MUST NOT be required to know any command syntax — no `COMMAND:` prefix, no rigid grammar. They send a clear natural-language message; the System is responsible for determining the intent. Any design that forces an end user to memorize syntax to be understood is a violation of this anchor.

(B) Three-tier intent resolution (the mandatory discipline). Every inbound subscriber message MUST be resolved to exactly one action via three tiers, in order — the first that succeeds wins:

```
TIER 1  command recognition — recognize the System's existing command set
from
                              natural language and map it to a structured
action
                              (a confident, deterministic match → that
action).
TIER 2  intent inference    — when NO command matches, infer the exact
intent from
                              the message (the LLM dispatch maps natural
language to
                              the right action) — NEVER guessing.
TIER 3  clarify (fallback)  — when neither a command nor a confident
intent can be
                              determined, REPLY to the original message,
TAG the
                              sender (`@username`, resolved via the
§11.4.104
                              IdentityResolver) and ask a PRECISE
clarifying question
                              that NAMES the candidate intents. No
guessing, no
                              silent drop.
```

Tier 3 is the anti-annoyance guarantee: the user is never ignored and never has to learn syntax — at worst they receive a friendly, specific "@user, did you mean X or Y?".

(C) Never guess, never drop. The System MUST NEVER guess an action — a wrong action is worse than a clarifying question (composes with §11.4.6 no-guessing). The System MUST NEVER silently drop or ignore an inbound message. We always do our best to determine the exact intent so end users are never annoyed; only genuine ambiguity reaches Tier 3, and Tier 3 always replies-tags-and-asks rather than failing silently.

(D) Anti-bluff (MANDATORY, composes with §11.4 / the end-user quality covenant). Every tier ships unit + integration + E2E + full-automation tests producing REAL captured evidence (no metadata-only / absence-of-error PASS, no false positive/negative): Tier 1 proven by a truth-table of natural-language messages → expected action+fields PLUS the conservative negatives that MUST fall through to "no match"; Tier 3 proven by an E2E ambiguous-message dispatch whose recorded reply body is EXACTLY `@<sender> <specific question>` (the

user is tagged + asked, not ignored) AND a NEGATIVE proving a clear command does NOT trigger clarify; a paired §1.1 mutation that breaks the recognizer's confidence guard (forcing a false-match) OR drops the clarify tag MUST FAIL a test. Evidence committed under `docs/qa/<run-id>/`.

Composes with §11.4 + §11.4.1..§11.4.16 (end-user quality / anti-bluff covenant — sub-rule (D) is bound by it), §11.4.6 (no-guessing — a wrong action is worse than a clarifying question), §11.4.104 (participant identity — the clarify reply tags the sender via the IdentityResolver), §11.4.5 / §11.4.69 (captured evidence), §11.4.98 (full-automation — every tier's test is self-driving), §1.1 (paired-mutation proof of the confidence guard + clarify tag).

Classification: universal (§11.4.17) — natural-language intent recognition with a three-tier recognize→infer→clarify discipline is a reusable interaction contract for ANY project that ships a messenger/command surface; the no-syntax mandate, never-guess rule, and reply-tag-and-ask fallback are vendor-neutral. Projects with NO messenger surface inherit the anchor latently (it binds the moment they ship one) — the §11.4.96 "principle binds even absent the surface" restatement pattern.

4-layer coverage per §11.4.4(b). Propagation gate `CM-COVENANT-114-105-PROPAGATION` enforces the literal anchor `11.4.105` across the canonical consumer fleet (parent + owned-submodule CLAUDE.md / AGENTS.md / QWEN.md). Paired §1.1 meta-test mutation strips the `11.4.105` literal from a consumer file → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.105 in the HelixConstitution submodule; detailed spec Herald `docs/design/INTENT_RECOGNITION.md`. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker. No escape hatch — no `--require-command-syntax`, `--guess-intent-ok`, `--skip-clarify`, `--drop-on-ambiguous` flag exists.

---

§ . . — Anti-bluff AV/test-validation techniques mandate (User-driven research, - -)

Short tag: `av-liveness-oracles`.

Forensic anchor — captured incidents (2026-06-02, genericised): a test asserting "media is playing on the target output" PASSed on the strength of a SINGLE captured frame that showed "a picture" — but the picture was a FROZEN / STALE frame left over from the previously-played content (a stale-producer / stuck-decoder failure mode), so the feature was broken for the end user while the test was green. A sibling incident: media briefly FLASHED on the WRONG output (the source/primary surface) for ~1 s before routing to the intended output, and no test sampled the wrong output during the transition window, so the glitch shipped undetected. A third class: a comparator (color / freeze / OCR oracle) that was itself broken — it PASSed on a deliberately-degraded artefact — meaning the analyzer, not just the feature, was the bluff. §11.4.5 mandates *captured* evidence and a baseline presence pass; §11.4.107 raises the bar to **liveness + correct-routing + self-validated-analyzer** so a single still frame, a wrong-output flash, or a broken oracle can no longer pass.

The mandate. Every test asserting that audio or video output is genuinely playing / advancing for the end user MUST satisfy ALL of the following — each is a §11.4 PASS-bluff to omit:

1. **A single captured frame is NOT proof of playback.** Prove LIVE, ADVANCING frames over a steady-state window (≥ N frames across ≥ a few seconds) via a **freeze-detection oracle** — a near-duplicate detector (the platform's frame-difference / `freezedetect`-class filter OR a perceptual-hash adjacent-frame distance), **NOT a byte-identical compare** (a re-encoded / 1-pixel-noisy stale frame defeats byte-identity; keep byte-identity only as a zero-cost early-out pre-filter). Adjacent frames MUST differ above a calibrated threshold; ≥ a sustained interval of near-identical frames ⇒ FROZEN ⇒ FAIL.
2. **An independent frame-advance counter from the platform's compositor / decoder telemetry** MUST increase across the window — an oracle in a DIFFERENT domain from the pixel check (compositor frame-timestamp advance, decoder `framesDecoded` / `framesRendered`, or equivalent). A flat counter over the window ⇒ stuck decoder ⇒ FAIL, even if the pixels appear to

move. Keeping a pixel-domain oracle AND a telemetry-domain oracle is metamorphic redundancy: a coordinated false-PASS now requires two independent failures.

3. **Loading / buffering is a DISTINCT state — wait for genuine playback-start before judging liveness.** Never false-FAIL a still-loading stream, never false-PASS a loading spinner. Poll the decoder frame-advance counter / first-advancing-frame / buffering flag until playback genuinely starts (generous per-class timeout). Timeout + content endpoint unreachable ⇒ SKIP-with-reason per §11.4.3 (`network_unreachable_external` / `geo_restricted`); timeout + reachable ⇒ FAIL with captured evidence. Only AFTER confirmed playback-start run the liveness checks over the steady-state window.
4. **Not-stale-from-previous cross-check.** Before asserting new content N is live, confirm its first frame DIFFERS from the previous content's last captured frame (perceptual-hash mismatch). Identical ⇒ the "new" content is a leftover frozen frame ⇒ FAIL. Critical for switch-between-items sequences.
5. **Measured FPS / no-lost-frames within tolerance** of the source's expected rate (where the source rate is knowable) or a sane floor otherwise; drop ratio above budget ⇒ FAIL (§11.4.5 frame-health).
6. **No-flash-on-the-wrong-output during a routing transition.** When content is meant to appear ONLY on a target output, sample the NON-target output at high frequency (e.g. every ~200 ms) across the transition window; ANY sampled wrong-output frame containing the content ⇒ FAIL (brief-flash glitch). (Where a content-protection regime legitimately forces a particular output, that regime is the expected case, not a fail — classify it explicitly, never by guessing per §11.4.6.)
7. **Drive tests through the realistic user path (feed / UI), not deep-link shortcuts.** Routing / transition / alternation bugs manifest on the path a real user takes (selecting from a feed / list via the UI); deep-link / direct-dispatch shortcuts bypass the very transition paths where these bugs live. Where the UI is not introspectable, fall back AND flag operator-attended per §11.4.52 — never fake-PASS. (This is the §11.4.48 UI-driven discipline applied to the liveness battery.)

8. **The oracle problem — use metamorphic relations when there is NO golden source.** For content the project does not own (streamed media), there is no golden reference, so full-reference comparison is impossible. Encode metamorphic relations instead: the SAME content captured on output-A vs output-B must be structurally equivalent (after resolution normalisation); a paused stream's frame-advance counter must STOP; a 2× speed must roughly double the frame-advance rate; the same clip played twice must produce equivalent liveness metrics. Each is a relation, not a fixed expected value — robust to content variety.

9. **Full-reference quality metrics vs a golden source for content the project DOES own.** Where a source reference exists (local-file playback of owned media, a test corpus), assert perceptual-quality / color fidelity (SSIM / VMAF / ΔE2000-class) against the temporally-aligned source so a pale / desaturated / wrong-range / soft-scaled render is caught across motion, not just at one sampled frame.

10. **Mutation-test every analyzer with a golden-good + golden-bad fixture pair** so the analyzer itself provably cannot bluff. For every comparator (freeze / motion / color / sharpness / aspect-ratio / FPS / OCR / audio-presence) keep a clean control fixture (must PASS) AND a deliberately-degraded fixture (must FAIL — frozen clip, black-gap clip, desaturated clip, downscaled clip, stretched clip, half-rate clip, overlay-baked frame, silent track). An analyzer that PASSes its golden-bad fixture is a bluff gate. This is the §1.1 paired-mutation discipline applied to the AV *analyzers*, not only the grep gates. Document any behaviourally-equivalent mutant expected NOT to flip so a survivor there is not a false alarm.

11. **Per-channel audio integrity, not a single aggregate number.** A single aggregate RMS misses a dead channel in a multichannel stream and a near-silent / DC-offset stream that scrapes past a bare floor. Assert per-channel RMS / loudness (an EBU-R128-class integrated/momentary loudness + per-channel level + silence-region census + DC-offset / NaN-Inf check); EVERY declared channel above floor. On-device, take an underrun / XRUN census from the audio subsystem — a starved pipeline is a defect even if the captured waveform "has sound".

12. **OCR overlay / subtitle detection needs a per-word confidence floor + a region-of-interest** to avoid BOTH a false-positive (noise read as a hostile overlay ⇒ false-FAIL of a working feature) AND a false-negative (a real overlay missed ⇒ false-PASS). Discard sub-confidence words; OCR the ROI where the text lives (bottom-third for subtitles, centre for dialogs); emit per-finding confidence so a reviewer sees WHY a finding fired. Where a subtitle must appear on one output and NOT another, assert both sides (a property relation per point 8).

13. **Thresholds MUST be calibrated on the project's own fixtures, not hardcoded from literature (§11.4.6 no-guessing).** Every freeze / motion / SSIM / VMAF / loudness / OCR-confidence threshold is recorded as captured evidence derived from a known-good and known-bad sample on the project's own capture path — never a literature constant asserted as fact. The capture path's own re-encode loss is part of the baseline.

Honest gaps (§11.4.6). True physically-displayed (photon) frame rate has no clean software oracle — the compositor / decoder counters measure the *presentation pipeline*, not photons at the sink; a high-speed external camera rig is the only ground truth. The counters are sufficient to catch a stuck / half-rate decoder and MUST be used, with the photon-FPS gap flagged honestly rather than claimed. Broadcast-QC vendor tooling validates that freeze / duplicate / lost-frame / black detection is industry-standard practice, but the exact proprietary thresholds are undocumented; the project adopts the published open-method analogues and calibrates per point 13.

4-layer coverage per §11.4.4(b). Per-fix: pre-build gate (e.g. a `CM-AV-LIVENESS-NO-FROZEN-FRAME`-class gate asserting every test that claims output-is-playing references the liveness battery — freeze oracle + frame-advance counter + not-stale cross-check + FPS — not a single-frame check; plus an analyzer-self-validation gate wiring the golden-good/golden-bad fixtures into the project's meta-test) + on-device / runtime test + **paired §1.1 meta-test mutation** (a test asserting output-is-playing with ONLY a single-frame check → gate FAILs; an analyzer that PASSes its golden-bad fixture → self-validation FAILs) + HelixQA Challenge entry for the user-visible feature. Every such PASS is emitted via `ab_pass_with_evidence` citing the motion / not-stale / frame-advance / fps / per-

channel-loudness / metamorphic / self-validation artefacts (feature classes `video_display`, `audio_output`, `subtitle_render` per §11.4.69) — NEVER a single screenshot.

Classification: universal (§11.4.17) — freeze-detection oracles, frame-advance telemetry, loading-aware playback-start waits, not-stale cross-checks, metamorphic relations, full-reference quality metrics, golden-good/golden-bad analyzer self-validation, per-channel loudness, and confidence-floored OCR are platform-neutral AV / test-validation techniques reusable by ANY project that validates media playback or any pixel/audio output; the project supplies the concrete capture mechanism (its compositor / decoder telemetry, its sink-introspection API per §11.4.13, its capture tool) and the calibrated thresholds per §11.4.35.

Composes with §11.4.5 (strict expansion of its video + audio quality bar to liveness + correct-routing), §11.4.6 (no-guessing — thresholds calibrated + captured, loading-vs-frozen disambiguated by evidence, content-protection regime classified not guessed), §11.4.50 (deterministic-consistency — the liveness battery runs N iterations identically), §11.4.68 (positive sink-side / downstream evidence — the frame-advance counter / sink-introspection cross-check is its video analogue), §11.4.69 (universal sink-side positive-evidence taxonomy — `video_display` / `audio_output` / `subtitle_render` classes), §11.4.85 (stress + chaos — liveness under sustained load + transition chaos), plus §11.4.2 (recorded evidence), §11.4.3 (topology SKIP-with-reason for unreachable content), §11.4.13 (sink-side introspection as one oracle), §11.4.48 (UI-driven path), §11.4.52 (autonomous validation; operator-attended fallback only when UI not introspectable), §1.1 (paired-mutation discipline applied to the analyzers).

Propagation. Propagation gate `CM-COVENANT-114-107-PROPAGATION` enforces the literal anchor `11.4.107` across the consumer fleet (parent + owned-submodule CLAUDE.md / AGENTS.md / QWEN.md); paired §1.1 meta-test mutation strips the `11.4.107` literal from a consumer file → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.107 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker. No escape hatch — no `--single-frame-proves-playback`, `--skip-liveness`, `--byte-identical-freeze-OK`, `--no-frame-advance-counter`, `--skip-not-stale-check`, `--allow-wrong-output-flash`, `--deep-link-shortcut-OK`, `--unvalidated-analyzer-OK`, `--aggregate-rms-suffices`, `--hardcoded-thresholds-OK` flag exists.

2026 06 03

§ . . — Four-layer fix-verification + runtime-signature-as-definition-of-done mandate (systematic-debugging Phase . , – –)

Short tag: `runtime-signature-definition-of-done`.

Forensic anchor (genericised, 2026-06-03). Across one batch, multiple fixes were "green" at every gate yet NOT working for the end user: a change present in the source tree and passed by both the pre-build (source) gate and the post-build gate never reached the boot-time kernel command-line embedded in the deployed boot artifact, so the corresponding output feature was dead despite all-green; sibling fixes correctly built into the system image were masked by stale per-user overlay copies left from a previous deployment (the running code was the stale overlay, not the freshly-built system code), so they merely *looked* broken; deployment did not produce a clean state (the mutable overlay was not wiped), so the stale shadow survived; and the validation suite ran against whatever was running — which could be the stale shadow — so it reported green on code that was never actually exercised. Per `superpowers:systematic-debugging` Phase 4.5: when each fix reveals a fresh "fixed-but-not-working" in a *different* place, that is NOT a series of independent bugs — it is ONE architectural VERIFICATION flaw, and patching each symptom is thrashing.

The architectural flaw (FACT, traced). A fix must cross FOUR distinct layers, and "fixed" at one layer does NOT imply fixed at the next: (1) **SOURCE** — the change is committed in the source file (what a grep-the-source pre-build gate checks); (2) **ARTIFACT** — the change's BYTES actually landed in the produced build artifact (image / bundle / installer / boot artifact / embedded command-line); (3) **RUNTIME-ON-CLEAN-TARGET** — the change is active on a *clean / fresh* deployment, the layer the end user experiences, with no stale overlay shadowing the deployed

code; (4) **USER-VISIBLE** — the feature actually works for the end user (the §11.4.5 / §11.4.69 captured-evidence layer). Green at layer 1 is the cheapest, least conclusive signal; nothing being green at layer 1 (or even layer 2) tells you about layers 3–4. A gate that verifies ONLY the source layer is insufficient and is itself a §11.4 bluff surface.

The mandate (ALL must hold).

1. **Runtime-signature-as-definition-of-done.** A fix is DONE only when its declared **runtime signature** is verified on a CLEAN / fresh deployment (the layer the end user experiences). Source-committed ≠ artifact-contains-it ≠ active-on-clean-target ≠ user-visible-working — these are FOUR distinct claims; proving one does NOT prove the next.
2. **Every fix declares ONE machine-checkable runtime signature** — a single observable on a clean target that proves the fix is BOTH active AND working (a property read from the running system, a downstream/sink-side report per §11.4.13, a captured-evidence assertion per §11.4.5 / §11.4.69, or a counter/state delta — never a re-grep of the source). The **registry of these per-fix runtime signatures is the SINGLE SOURCE OF TRUTH for "fixed"** — it REPLACES "a gate greps the source" as the definition of done.
3. **Verification gates MUST span all four layers**, per fix: **source** (pre-build / pre-merge), **artifact** (post-build — assert the change's BYTES landed in the produced artifact, NOT merely that the source still contains the change; cf. the embedded-command-line / stripped-asset failure mode), **runtime-on-clean-target** (post-deploy — assert the runtime signature on a freshly-deployed clean target), **user-visible** (captured evidence per §11.4.5 / §11.4.69). A gate that asserts only the source layer is insufficient and is itself a §11.4 bluff surface.
4. **Eliminate the stale-deployment / shadow layer — by construction.** Deployment MUST yield a CLEAN state (wipe the mutable overlay / per-user mutable state that can shadow the deployed build) **OR** a pre-validation assertion MUST prove `running-artifact == built-artifact` (no stale override masks the freshly-deployed build) BEFORE any validation runs. Validation that runs against possibly-stale deployed state is INVALID and any PASS it produces is a §11.4 PASS-bluff — the test exercised code that was never actually deployed.

5. **Meta-rule (composes §11.4.102 Phase 4.5).** ≥ 3 "fixed-but-not-working" discoveries within a single cycle are the signal of an architectural VERIFICATION flaw, NOT three independent product bugs. On the 3rd such discovery the agent MUST STOP patching symptoms (per §11.4.4 test-interrupt-on-discovery), fix the VERIFICATION pipeline itself (close whichever of the four layers is unverified), and re-certify EVERY item in the batch through the corrected pipeline on a clean target.
6. **A batch is "validated" only after COMPREHENSIVE per-item runtime-signature verification on a clean baseline** — NOT after the batch-touched items' own tests pass. Spot-validating only the touched items (per §11.4.40's full-suite reasoning) misses the cross-layer gap: a touched item can be source-green while its runtime signature is dead on a clean target.
7. **Machine-created evidence at every layer (extension, 2026-08-15 — Point 8 of the 9-point mandate).** At each of the four verification layers (SOURCE / ARTIFACT / RUNTIME-ON-CLEAN-TARGET / USER-VISIBLE), the layer's PASS MUST cite MACHINE-CREATED, MACHINE-VERIFIABLE evidence PRODUCED AT THAT LAYER by the layer's own gate — a schema-parseable file (static-analysis JSON + review verdict at SOURCE; build log + fingerprint checksum + byte-check at ARTIFACT; on-target verdict + runtime-signature + telemetry at RUNTIME; OCR/vision/audio/sink-probe oracle output at USER-VISIBLE) — cited by path + sha256 + timestamp per §11.4.207. A PASS at a higher layer that cites evidence from a lower layer ALONE is a §11.4.226 wrong-layer violation and does NOT satisfy the higher layer's obligation. Every evidence analyzer is §11.4.107(10) golden-good / golden-bad / negative-control validated; an analyzer that passes its golden-bad fixture voids every verdict it ever emitted (§11.4.201 self-validation). Operator eyeballing, narrative-only PASS, and prediction of success are FORBIDDEN — §11.4.262 binds this at every gate. Propagation: the existing `CM-COVENANT-114-108-PROPAGATION` stays unchanged (literal `11.4.108` unchanged, stays GREEN); `CM-RUNTIME-SIGNATURE-REGISTRY` is EXTENDED with the per-layer machine-evidence-file-schema check; a new paired mutation that forges a runtime PASS whose only evidence is a source-grep transcript → the gate FAILs.

Classification: universal (§11.4.17) — the four-layer source→artifact→runtime-on-clean-target→user-visible verification gap, the per-fix runtime-signature registry as the definition of done, the artifact-byte-presence gate, the clean-deployment-or-

equal-artifact pre-validation assertion, and the ≥3-discoveries-means-architectural-flaw meta-rule are platform-neutral verification-pipeline disciplines reusable by ANY project that builds an artifact, deploys it, and validates the deployed result; the consuming project supplies the concrete artifact format, its clean-deployment / equal-artifact mechanism, and its per-fix runtime-signature observables per §11.4.35.

Composes with §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded evidence — the runtime signature IS the recorded evidence), §11.4.4 (test-interrupt-on-discovery + retest-from-clean-baseline — clause 5's STOP-and-fix-the-pipeline is its §11.4.108 specialisation; "clean baseline" is exactly clause 4's clean deployment), §11.4.5 (captured-evidence quality — the user-visible layer), §11.4.6 (no-guessing — every layer asserted by captured evidence, never assumed to have propagated), §11.4.27 (no-fakes-beyond-unit — runtime signatures exercise the real deployed system), §11.4.40 (full-suite retest — §11.4.108 adds the cross-layer per-item runtime-signature dimension clause 6 demands), §11.4.46 (validate-recent-work-before-full-suite — the clean-baseline / equal-artifact pre-flight is its §11.4.108 pre-condition), §11.4.50 (deterministic consistency — the runtime signature verifies identically across N iterations), §11.4.52 (autonomous-validation — runtime signatures are machine-checkable without an operator), §11.4.69 (universal sink-side positive-evidence taxonomy — a runtime signature is a taxonomy-class observable), §11.4.102 (systematic-debugging — Phase 4.5 architectural-flaw recognition IS clause 5's trigger).

Propagation. Propagation gate `CM-COVENANT-114-108-PROPAGATION` enforces the literal anchor `11.4.108` across the consumer fleet (parent + owned-submodule CLAUDE.md / AGENTS.md / QWEN.md); paired §1.1 meta-test mutation strips the `11.4.108` literal from a consumer file → the gate FAILs. A recommended per-fix gate `CM-RUNTIME-SIGNATURE-REGISTRY` asserts every fix declares a runtime signature verified at the artifact + runtime-on-clean-target layers (not only the source layer); paired §1.1 mutation downgrades a fix's verification to source-only → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.108 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker. No escape hatch — no `--source-green-is-done`, `--skip-artifact-byte-check`, `--validate-against-running-state`, `--no-clean-deployment`, `--skip-runtime-signature`, `--spot-validate-touched-only` flag exists.

2026 06 03

§ . . — Pre-build build-readiness verdict + change-impact clash detection mandate (operator mandate, - -)

Short tag: `pre-build-readiness-verdict`.

§-slot history note. Drafted as §11.4.109 in `docs/research/prebuild_rigor_20260603/PROPOSED_CONSTITUTION_ANCHOR.md`, renumbered to §11.4.110 per §11.4.71 fetch-before-push after the concurrent landing of §11.4.109 (Anti-Forgetting Enforcement, commit `1d9e5d6`). Same collision-resolution pattern as §11.4.75 / §11.4.76.

Forensic anchor (genericised, 2026-06-03). A fix shipped a new system-property *read* in a code/init change but introduced no matching security-policy grant and no property-context type entry for it — so the read was *silently denied* at runtime and the feature was dead, while every pre-build gate stayed green because the gate only grepped the source file for the change. The defect class generalises: a change introduces a new dependency on a *second* artifact (security policy, context file, service registry, interface freeze-snapshot, symbol table, build-graph node) that the pre-build never cross-checks, so the change is "ready" by every existing gate yet broken the instant it runs. This is the SOURCE→ARTIFACT half of §11.4.108's four-layer gap, shifted *left* to pre-build time: most of these clashes are **statically catchable from the change diff itself, before any build**.

The architectural flaw (FACT, traced). Pre-build gates that validate each artifact *in isolation* (the policy file is internally consistent; the freeze-snapshot is well-formed) do NOT cross-check that a change introducing a new dependency *also* introduced the matching second artifact. The cross-check is **diff-driven** (its input is the change set, not any single file) and is therefore a capability no per-file gate provides. Without it, the only protection is author-vigilance ("remember to also add the policy line"), which §11.4.75 already established is the failure mode that mechanical enforcement exists to replace.

The mandate (ALL must hold).

1. **A single deterministic READY-FOR-BUILD verdict is mandatory.** Before any artifact rebuild, the project MUST emit ONE machine-checkable verdict — *the codebase IS / IS NOT ready for build* — backed by a captured-evidence bundle enumerating every gate family's status. The rebuild orchestrator MUST refuse to start the build unless the verdict is READY (non-zero exit on NOT-READY). A green individual gate is necessary but not the verdict; the verdict is the aggregate function and cannot report READY while any family reports FAIL.
2. **A diff-driven change-impact + conflict/clash detector is mandatory.** For every change in the batch, the pre-build MUST cross-check that any newly-introduced dependency on a *second* artifact is satisfied — at minimum: a new system-property read ⇄ the property is typed in the platform's property-context registry AND the reading domain has the matching read-grant; a new registered service ⇄ its service-context entry exists; a new declared startup/init service ⇄ its security label is present; a new/changed stable interface ⇄ its freeze-snapshot/version was updated in the same change; a new native-library dependency ⇄ the named library resolves to a known module or prebuilt; a new security-policy rule ⇄ every type/attribute it uses is defined; and two changes in the batch touching the SAME function / resource / configuration seam ⇄ the collision is explicitly acknowledged (the cross-fix-interaction class). Each sub-detector is independently anti-bluff (clause 5).
3. **Coverage-completeness is a gate, not a convention.** Every changed source file in the batch MUST map to ≥ 1 pre-build gate + ≥ 1 deployed-target test + ≥ 1 paired §1.1 mutation (the four-layer §11.4.4(b) floor). A baseline file records the legacy tail of known holes; the gate FAILs on any NEW hole and the covered-ratio ratchets upward over phases per §11.4.50. A change with no owning gate/ test/mutation is a coverage hole and blocks the verdict.
4. **Two-speed honesty — grep-speed vs REQUIRES_BUILD — is mandatory.** The regime MUST separate always-on grep-speed gates (run every commit, seconds) from REQUIRES_BUILD heavy gates (the build-graph parse-only dry-run; full security-policy neverallow compilation; built-symbol ABI diff). Heavy gates run as diff-gated opt-in stages (triggered only when the change touches build-graph / policy / interface files) and MUST run bounded per the host-

session-safety budget (§12.6/§12.7) or inside the project's containerized build. Conflating the two — making the grep path slow, or claiming grep proves graph/neverallow correctness — is forbidden per §11.4.6.

5. **Every gate and analyzer is anti-bluff by paired mutation.** Each gate family AND each wired third-party analyzer (its exit-code-on-failure semantics included) MUST ship a paired §1.1 mutation that plants the exact defect class and asserts the gate FAILs (e.g. a property read with no policy → the clash detector FAILs; a syntactically-broken build-graph fragment → the dry-run reports FAIL; an analyzer that exits 0 on a planted violation is itself the bluff the mutation catches). An analyzer wired without its mutation pair is a §11.4 bluff surface.

6. **Honest boundary — pre-build catches statically-catchable defects only.** A READY verdict proves the change is internally consistent and ready to build; it does NOT prove the feature works on the deployed target. The correct claim is: *after the pre-build passes, the residual defects deployed-target / manual testing can find are runtime/hardware defects, NOT statically-catchable (preventable) ones* — the regime makes the **preventable** class empty, not the **all-defects** class. Overclaiming that manual testing will find nothing is itself a §11.4.6 violation. Build-graph correctness, full neverallow safety, and ABI correctness are proven ONLY by the REQUIRES_BUILD families (clause 4); the grep tier proves harness-presence + symbol-definedness, never full correctness. Runtime/USER-VISIBLE correctness remains entirely §11.4.108's four-layer job.

Classification: universal (§11.4.17) — the single ready-for-build verdict, the diff-driven change-impact/clash detector, the coverage-completeness gate, the grep-speed-vs-REQUIRES_BUILD two-speed split, the per-gate paired mutation, and the statically-catchable-only honest boundary are platform-neutral pre-build-rigor disciplines reusable by ANY project that builds an artifact from source; the consuming project supplies its concrete property/service/policy registries, its build-graph parse-only command, its interface-freeze mechanism, and its changed-file-to-gate mapping per §11.4.35.

Composes with §11.4.1 (FAIL-bluffs forbidden — a clash-detector that crashes on a malformed diff and FAILs is as misleading as one that passes silently), §11.4.4 (test-interrupt-on-discovery — a NOT-READY verdict triggers STOP-and-fix before the build), §11.4.6 (no-guessing — the two-speed honest boundary; the grep tier MUST NOT claim correctness it cannot prove), §11.4.9 (batch-source-fixes-before-

rebuild — the verdict gates exactly that rebuild), §11.4.27 (no-fakes-beyond-unit — wired analyzers exercise the real change), §11.4.50 (deterministic consistency — the coverage-completeness ratchet shares §11.4.50's threshold mechanism and N-iteration determinism), §11.4.67 (target-shell-parseability — the clash-detector helper itself parses under its target shell), §11.4.75 (mechanical enforcement — §11.4.110 is the mechanical replacement for author-vigilance on the clash class), §11.4.92 (multi-pass change-evaluation — the verdict is the mechanical companion to Pass 1/2/3), §11.4.108 (four-layer fix-verification — §11.4.110 is the SOURCE→ARTIFACT half shifted left to pre-build time; §11.4.108 owns the RUNTIME-ON-CLEAN-TARGET→USER-VISIBLE half; together they span all four layers).

Propagation. Propagation gate `CM-COVENANT-114-110-PROPAGATION` enforces the literal anchor `11.4.110` across the consumer fleet (parent + owned-submodule CLAUDE.md / AGENTS.md / QWEN.md); paired §1.1 meta-test mutation strips the `11.4.110` literal from a consumer file → the gate FAILs. Recommended per-family gates `CM-READY-FOR-BUILD-VERDICT` (single deterministic verdict gates the rebuild), `CM-CHANGE-IMPACT-CLASH-DETECTOR` (diff-driven second-artifact cross-check), `CM-COVERAGE-COMPLETENESS-GATE` (changed-file ⇄ gate+test+mutation with baseline ratchet), `CM-BUILDGRAPH-DRYRUN-WIRED` + `CM-SEPOLICY-NEVERALLOW-WIRED` (REQUIRES_BUILD opt-in stages exist + bounded-safe); each with a paired §1.1 mutation per clause 5. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.110 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--source-green-is-ready`, `--skip-clash-detector`, `--skip-coverage-gate`, `--no-ready-verdict`, `--grep-proves-neverallow`, `--skip-buildgraph-dryrun`, `--build-without-ready-verdict` flag exists.

§ . . — Rock-solid-proof-or-deep-research mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-03):

> "Every single reported issue MUST BE fully and 100% validated with rock solid proofs! Nothing can be considered fixed or completed without hard evidence! No false results or bluff(s) of any kind is allowed! If we are not sure on how to achieve full testing, validation and verification of something we MUST ALWAYS perform deep web research for all possible data (articles, documentation, guides, and other resources) and opensourced codebases which we can use to solve our problems and perform testing with validation and verification which produces rock-solid evidence(s) and leaves no space for false results or any kind of bluff!"

Forensic case study (FACT — captured). In the 1.1.8-dev remediation cycle the validation method for two feature classes was, at first, genuinely unclear: relocating a `FLAG_SECURE` secure surface to a secondary display (whose pixel capture returns black) and asserting on-screen content in non-introspectable streaming-app UIs (whose accessibility hierarchy is blank). Rather than declaring those classes "untestable" or accepting a metadata-only PASS, the cycle performed deep web research (`docs/research/testing_frameworks_20260603/`) into platform/HDCP behaviour, computer-vision template-matching, ROI OCR with confidence floors, freeze-detection / frame-advance liveness oracles, and network-side sink probes. That research yielded the CV/OCR/liveness/sink-probe oracle stack (now §11.4.107 + §11.4.112 + §11.4.117) that made rock-solid, evidence-producing validation possible where it had appeared impossible. The lesson — "unclear how to validate" is a research trigger, NEVER a bluff licence — is now constitutional.

The mandate. Every single reported issue, every fix, and every claimed completion MUST be fully and 100% validated with rock-solid CAPTURED proof per §11.4.5 / §11.4.69 / §11.4.107 before it may be marked fixed / implemented / completed (the §11.4.33 terminal closure vocabulary). Hard, captured evidence is the bar — nothing may be considered fixed or complete without it. Metadata-only PASS, configuration-only PASS, absence-of-error PASS, and grep-without-runtime PASS are all forbidden (§11.4 / §11.4.1) — no false results and no bluff of any kind, at any layer, are allowed.

The research-or-don't-bluff rule (the operative addition). When the agent is UNSURE how to fully test, validate, or verify something — when no obvious evidence-producing method exists, OR the candidate method would yield only

metadata/config/absence-of-error evidence (and is therefore a bluff per §11.4.1) — the agent MUST ALWAYS first perform deep web research per §11.4.8 + §11.4.99 (official documentation, technical articles, guides, vendor references, standards, issue trackers, and open-source codebases the project can reuse or adapt) to DISCOVER or BUILD a validation method that produces rock-solid evidence and leaves no space for a false result. Declaring something "untestable", "not automatable", or accepting a metadata-only PASS WITHOUT having first exhausted this deep-research path is itself a §11.4.123 violation — the same severity class as a PASS-bluff. The research output (cited source URLs + the evidence-producing method it yielded, OR the literal "NO external solution found — original work" per §11.4.8) is the captured proof that the path was exhausted. Only after that research genuinely fails to yield any evidence-producing method may the item be classified `PENDING_FORENSICS:` / `Operator-blocked` (§11.4.21) / `structurally-impossible` won't-fix (§11.4.112) — with the cited research as the evidence that the classification is earned, never a convenience.

Composition. §11.4.123 composes with §11.4.5 (captured-evidence quality is the proof shape), §11.4.6 (no-guessing — "probably fixed" without captured evidence is forbidden; the closure must be FACT), §11.4.8 (deep-web-research-before-implementation — §11.4.123 extends it from "before designing a fix" to "whenever the VALIDATION method is unclear"), §11.4.52 (autonomous-validation — the research must yield an autonomous, evidence-producing path wherever one is discoverable), §11.4.69 (universal sink-side positive-evidence taxonomy — every PASS cites a taxonomy-class artefact), §11.4.99 (latest-source verification of the discovered method), §11.4.107 (the AV/test-validation oracle techniques the 1.1.8-dev research yielded), §11.4.118 (discovery-pressure — the same "absence of evidence of looking is not evidence of absence" principle applied to per-issue validation). It also composes with §11.4.21 / §11.4.112 (the earned escape valves — reachable ONLY after the deep-research path is exhausted with cited evidence).

Enforcement. Propagation gate `CM-COVENANT-114-123-PROPAGATION` enforces the literal anchor `11.4.123` across the consumer fleet (every CLAUDE.md / AGENTS.md / QWEN.md). Recommended gate `CM-ROCK-SOLID-PROOF-OR-RESEARCH` (a closure-audit detector that flags any item closed `Fixed` / `Implemented` / `Completed` whose tracker entry lacks a captured-evidence artefact path per §11.4.69, AND any `PENDING_FORENSICS:` / `Operator-blocked` / `structurally-impossible` classification whose entry lacks a cited deep-research trail per §11.4.8/§11.4.99). Paired §1.1 meta-test mutations: strip the

`11.4.123` literal → propagation gate FAILs; close an item with a metadata-only PASS and no captured-evidence path → `CM-ROCK-SOLID-PROOF-OR-RESEARCH` FAILs; classify an item `Operator-blocked` with no cited research trail → the same gate FAILs. (Gate-code implementation lands as a separate follow-up work item; this anchor defines the contract.)

Classification: universal (§11.4.17) — "every closure carries rock-solid captured proof, and unclear-how-to-validate triggers deep research rather than a bluff" is a platform-neutral discipline reusable by ANY project; the consuming project supplies its concrete capture mechanisms + research corpora per §11.4.35.

Canonical authority: this Constitution.md §11.4.123 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--metadata-pass-suffices` (11.4.139), `--skip-proof`, `--untestable-without-research`, `--config-only-closure-OK`, `--bluff-when-unsure` flag exists. (2026-06-08)

§ . . — Fresh-process clean-artifact runtime-signature mandate (User mandate, - -)

Refines §11.4.108. Before any post-deploy validation — ESPECIALLY a non-pixel proxy verification (the subdebug/accessibility-cue channel used for FLAG_SECURE displays) — the harness MUST assert running-artifact == built-artifact: the deploy yielded a CLEAN target (mutable-overlay/userdata wiped) OR a pre-validation check proves no stale overlay shadows the deployed code (e.g. every guarded package — incl. the Presenter that emits the subtitle cue — resolves to the system partition, no per-user override). A stale shadow of the cue-emitting component (e.g. a Presenter APK predating the denylist) makes the proxy report on code that was never deployed — any PASS is a §11.4 PASS-bluff. Each fix declares ONE machine-checkable runtime signature verified on the clean target (the §11.4.108 registry IS the definition of done); for the subtitle class the signature is "the shipped Presenter APK contains the denylist literal (case-insensitive) AND the subdebug channel emits `candidate REJECTED reason=chrome-label` for a menu label."

Classification: universal (§11.4.17). Composes §11.4.46 / §11.4.108 / §11.4.130 / §11.4.135 / §11.4.137. Recommended gate `CM-CLEAN-ARTIFACT-RUNTIME-SIGNATURE` + `CM-COVENANT-114-139-PROPAGATION` (literal `11.4.139`) + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.139. Non-compliance is a release blocker. No escape hatch — no `--validate-against-running-state`, `--skip-clean-precondition`, `--shadow-OK` flag.

2026 06 10

§ . . — Reproduce-first test + same-test-confirms-fix + mandatory extend-to-all-cases workflow (User mandate, - -)

Forensic anchor — verbatim operator intent (2026-06-10): every reported problem MUST be handled by a NAMED three-step test workflow — (1) FIRST author a test that **confirms + reproduces** the reported problem AND is **used as an investigation instrument** to obtain additional forensic data characterising the defect; (2) after the fix, use the **SAME test** to confirm the problem is GONE; (3) THEN **extend** the tests to cover ALL flows, cases, edge cases, and the variety of potential situations for that functionality, confirming no issue of any kind — with every step producing rock-solid PHYSICAL captured evidence, no false results, no bluff.

§11.4.146 does NOT re-author its component disciplines; it NAMES and BINDS them into the operator's exact per-defect sequence and adds ONLY the two emphases the components leave implicit. The mandate (ALL hold):

(STEP 1 — REPRODUCE-FIRST + INVESTIGATE). Before any fix, author the §11.4.43 RED test as a §11.4.115 RED-baseline-on-the-broken-artifact (reproduce the defect on the CURRENT pre-fix artifact, capture defect-present physical evidence per §11.4.5/§11.4.69/§11.4.107). **NEW EMPHASIS (D1):** that same RED test is ALSO a deliberate **investigation instrument** per §11.4.102(A) — it MUST be used to gather ADDITIONAL forensic data characterising the defect (its triggers, boundaries, input/topology scope, adjacent failure modes), and that characterisation MUST feed BOTH the §11.4.102 root-cause/fix design AND the STEP-3 extend scope. A RED test used only as a binary present/absent gate, with no characterisation captured, satisfies §11.4.115 but NOT §11.4.146's STEP 1.

(STEP 2 — SAME-TEST-CONFIRMS-FIX). After the fix, the SAME test source (the §11.4.115 polarity switch flipped `RED_MODE=1→0`) confirms the defect is ABSENT — RED-on-broken then GREEN-on-fixed, both captured, on a clean target per §11.4.108/§11.4.139, validated-first-after-redeploy per §11.4.130, deterministic per §11.4.50. No separate happy-path test substitutes for the polarity-flipped reproduction test (the §11.4.43/§11.4.115 PASS-bluff).

(STEP 3 — EXTEND-TO-ALL-CASES, mandatory per-fix). NEW EMPHASIS (D2): immediately after STEP 2 confirms the fix, the test MUST be **fanned out across the full case-space of the SAME functionality** — all flows, valid + invalid + boundary (§11.4.85 stress empty/max/off-by-one) + concurrent (§11.4.85 contention) + failure-injection (§11.4.85 chaos) + topology variants (§11.4.3) — confirming no issue of any kind, with PROVABLE enumerated coverage per §11.4.118 (a listed case-set with per-case outcome, never "no other issues found"). This is a REQUIRED step of the per-defect workflow, NOT deferred to a release-cycle discovery sweep and NOT reduced to a single guard. Each fanned-out case that exercises a user-visible path carries rock-solid physical evidence per §11.4.123; each is registered into the §11.4.135 standing regression-guard suite so the functionality's full case-space is permanently guarded; a newly-discovered defect in the fan-out triggers §11.4.4 test-interrupt + re-enters STEP 1 for that new defect.

(ANTI-BLUFF, all steps). Every step's PASS is rock-solid CAPTURED physical evidence per §11.4.123 (§11.4.5/§11.4.69/§11.4.107) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/§11.4.1); when STEP 1 or STEP 3 validation method is unclear, deep-research-before-declaring-untestable per §11.4.123/§11.4.8/§11.4.99. An operator-found defect the workflow's green suite missed triggers §11.4.138 (bluff-audit + permanent guard).

Honest boundary (§11.4.6): STEP 3 reduces the functionality's unknown-unknown surface; it does not prove zero remaining defects (no finite case-set can, per §11.4.118) — the earned claim is "the reproduce→confirm pair + the enumerated extend case-set are all green with captured proof," un-exercised cases stated as honest gaps, never silently implied clean.

Classification: universal (§11.4.17) — a platform-neutral defect-lifecycle workflow reusable by ANY project; the consuming project supplies its reproduction/polarity harness, case-space enumeration, and capture mechanism per §11.4.35. Composes §11.4.43 (TDD RED step) / §11.4.115 (RED-baseline + polarity — STEP

1+2 core) / §11.4.102 (systematic-debugging — STEP 1 investigation) / §11.4.130 (validate-fix-first — STEP 2) / §11.4.108 + §11.4.139 (clean-target runtime signature) / §11.4.50 (deterministic) / §11.4.85 (stress+chaos case classes — STEP 3) / §11.4.118 (enumerated discovery coverage — STEP 3) / §11.4.135 (permanent regression guard — STEP 3 registration) / §11.4.3 (topology variants) / §11.4.123 (rock-solid-proof — all steps) / §11.4.5 / §11.4.69 / §11.4.107 (physical evidence) / §11.4.138 (operator-escape) / §11.4.4 (test-interrupt) / §107 / §1.1. Propagation gate `CM-COVENANT-114-146-PROPAGATION` (literal `11.4.146` across the consumer fleet) + recommended gate `CM-REPRODUCE-FIRST-THEN-EXTEND` (every closed defect's fix carries: a §11.4.115 reproduce-first polarity test with captured defect-characterisation [STEP 1], its `RED_MODE=0` GREEN confirmation [STEP 2], AND an enumerated per-functionality extend case-set registered into the §11.4.135 suite [STEP 3] — a closure with the reproduce→confirm pair but NO enumerated extend case-set FAILs) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; close a defect with only the reproduce→confirm pair and no extend-case-set → `CM-REPRODUCE-FIRST-THEN-EXTEND` FAILs; gate-code = separate work item).

(D3) STATUS-CUSTODY mechanical binding — a done-claiming status write is REFUSED without the workflow's machine-written evidence chain (extension, research-derived, 2026-07-17). Forensic FACT (genericised, 2026-07-17): in a consuming project this workflow and §11.4.115 already said exactly this — as prose — and did not bind: of 576 tracked items, 182 claimed a done/ready status and **173 (95%) had NO registered guard at all**; an operator manually sampled 6 done-claiming items and found **6/6 broken** — with 95% unguarded, the *expected draw*, not bad luck; one item recorded 4 Reopened events and 0 terminal events (its status moved with no custody trail), and recurrences re-entered the tracker as NEW ids so even the 52% recorded reopen rate undercounts. A rule that lives only in prose is re-learned per session and skipped under pressure; a further prose restatement would be the disease's next instance — the deliverable is the BINDING, not more words. Therefore (ALL hold): (1) **the custody chain is a file chain a script checks, with prose nowhere in it** — `item status ∈ done/ready ⇒ a guard-registry row keyed by the item's EXACT stable id (legacy/alias keys resolved via a checked-in alias map — a guard keyed to a non-item id fails the join and satisfies nothing) ⇒ an executable registered guard ⇒ the §11.4.115(F) machine-written RED+GREEN verdict pair ⇒ evidence files matching the item's feature-class shape` ; (2)

Seam A — the status write itself is refused: the workable-items engine (§11.4.93) refuses a transition to any done/ready status while the item's chain is incomplete; the engine change is project-agnostic per §11.4.28 — it reads a consumer-owned custody configuration, and every project supplies its registry / verdict-store / class-map paths as DATA per §11.4.35; (3) **Seam B — a full-table sweep gate, not a diff check**: on every pre-build run, EVERY done/ready item in the SSoT (minus the ratchet) must hold a complete chain, so an out-of-band raw-DB write that bypasses Seam A is caught at the next sweep rather than never; (4) **Seam C — the release seam** per the §11.4.135 verdict-coverage extension (ABSENCE of a verdict for the release-candidate artifact blocks exactly as a FAIL does); (5) **§11.4.205 construction order** — the FIRST artifact to land is the executable sweep + its golden-bad meta-fixture (a temporary SSoT overlay holding a terminal item with NO custody chain; the meta-test RUNS the sweep against it and requires FAIL — grep-only meta-assertions forbidden per §11.4.201/ §11.4.107(10) — and ALSO asserts the sweep is WIRED: de-wiring its invocation from the pre-build runner flips the meta-test RED, because "installed" is a verified runtime state per §11.4.205(3)); governance text citing the gate lands only in the same commit as the executable, so documenting this clause without implementing the hook is itself caught — the acceptance check is one command: run the sweep; (6) **adoption ratchet** — per the §11.4.135 extension, the pre-existing done-claiming-unguarded backlog is snapshotted once, monotone decrease only. Honest boundary (§11.4.6): the binding proves that no status outruns its evidence and that the REPORTED defect cannot silently return under the guard's conditions — it does NOT prove the feature bug-free (family dedup per §11.4.186 remains), does NOT make a miscalibrated oracle honest (§11.4.107(10) goldens are necessary, not sufficient), does NOT replace the §11.4.185 manual-QA sufficiency gate, and makes target availability a HARD dependency of status progress BY DESIGN — a visible block replaces a silent pass. Recommended gate `CM-STATUS-CUSTODY` (Seam-A refusal present in the engine + Seam-B full-table sweep wired + ratchet monotone-decreasing) + paired §1.1 mutation (mark a chain-less item done via a raw DB write → the sweep FAILs; de-wire the sweep → its meta-test FAILs; gate-code = separate work item). Composes §11.4.15 / §11.4.28 / §11.4.33 / §11.4.35 / §11.4.54 / §11.4.93 / §11.4.95 / §11.4.106 / §11.4.115(F) / §11.4.135 / §11.4.185 / §11.4.186 / §11.4.201 / §11.4.205.

Canonical authority: constitution submodule `Constitution.md` §11.4.146. Non-compliance is a release blocker. No escape hatch — no `--skip-reproduce-first`, `--fix-without-red`, `--skip-extend-to-all-cases`, `--reproduce-confirm-suffices`, `--defer-extend-to-release-sweep`, `--single-guard-suffices`, `--no-defect-characterisation` flag — and, per (D3), no `--status-without-custody`, `--prose-closure-suffices`, `--skip-custody-sweep`, `--raw-db-status-write-OK` flag.

2026 06 16

§ . . — Intensive all-feature/flow/edge-case video-recording + read-the-screen content-verification mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-16): "Add the following mandatory rules / guidelines / mandatory constraints ... regarding the recording of all features, flows, cases, edge cases and full validation and verification of it all with real results, covered with evidence and rock solid physical proofs and no bluffs of any kind! All projects MUST BE covered with such intensive testing and video recordings produced as the results! Default save path of them all is ALWAYS to be Downloads directory under host machine's user's home directory! All video recordings MUST contain valid working features with no false results showed in it and all printed out logs, messages, UI labels, dialogs, toasts MUST BE really read by the testing System (HelixQA MUST BE used!!!) so there are no issues or bluff!"

Forensic case study (FACT, 2026-06-16): the §11.4.153 feature-video sweep recorded `cli -stream` and the recording SHOWED `streaming generation failed: invalid character 'd' looking for beginning of value` — a REAL failure the sweep caught precisely because the on-screen text was READ + analyzed, not merely "a video was produced". Root cause was a §11.4.108 stale binary (source-fixed at commit 74077736, deployed artifact stale); the recording-that-reads-the-screen turned an otherwise-invisible artifact gap into a caught defect (then fixed + §11.4.135-guarded). This is the value §11.4.158 makes universal: a recording is proof ONLY when its shown content is machine-read and verified to be a genuine working result.

Strict intensification of §11.4.153 (per-feature Status+video ledger) + §11.4.107 (liveness + analyzer self-validation) + §11.4.117 (CV/OCR pixel oracle) + §11.4.137 (content-correctness oracle) + §11.4.27/§11.4.52 (HelixQA + autonomous validation), binding them into ONE intensive-coverage record-and-read mandate. ALL hold:

(A) Intensive total coverage. Every project MUST be covered by intensive automated testing that EXERCISES + RECORDS every feature, every flow, every use case, AND every edge case (valid / invalid / boundary / empty / max / concurrent / failure-injection per §11.4.85) — not a happy-path sample. A feature/flow/edge-case with no recorded, read-verified evidence is a §11.4 PASS-bluff at the coverage layer (composes §11.4.25 full-automation-coverage + §11.4.118 discovery-completeness + §11.4.153 per-feature ledger).

(B) Real results, rock-solid physical proof, zero bluff. Every recording MUST show the feature GENUINELY WORKING with REAL results (real prompts→real LLM/service responses→real outputs per §11.4.153) — NO false / simulated / stale / frozen result may appear in any recording (§11.4.2/.5/.107). A recording that shows an error, a bluff response, a frozen/stale frame, or a non-working feature is a FINDING (→ §11.4.153(4) / §11.4.4 fix→retest→re-record), NEVER a confirmation.

(C) Read-the-screen content verification (HelixQA-driven). The testing System MUST ACTUALLY READ the content shown in each recording — every printed log line, message, UI label, dialog, toast, status text — and VERIFY it (OCR/ROI per §11.4.117 / §11.4.137(12) with a confidence floor for pixel surfaces; direct text capture for terminal/log surfaces; the §11.4.107(10) golden-good/golden-bad self-validated analyzer). "A video was produced" is NOT evidence; "the System read the on-screen text and confirmed it is a genuine working result" is. HelixQA (§11.4.27, the HelixDevelopment/HelixQA submodule) MUST be the driver/orchestrator of this exercise→record→read→score pass — its banks exercise the features, capture the recordings, read the shown content, and score PASS only on a read-confirmed genuine result with the captured artefact path (§11.4.69).

(D) Default save path = `$HOME/Downloads`. Unless a project declares an explicit override per §11.4.35, the default save path for ALL recordings is the host user's home Downloads directory (`$HOME/Downloads` — resolved from the running user's home at runtime, NEVER hardcoded). A project MAY override the location per §11.4.35 (e.g. an external-drive recordings root); the override is recorded in the project layer, not here. Filenames carry the §11.4.155 project-name prefix; window-

scoped capture + fresh-corpus rotation per §11.4.154; raw corpus git-ignored + code-intelligence-excluded per §11.4.128, only curated evidence committed under `docs/qa/<run-id>/` (§11.4.83) at release prep.

(E) No false results survive. Composes the whole §11.4 anti-bluff covenant — a recording-confirmed PASS the operator (or a later re-read) contradicts is a §11.4.138 operator-escape → mandatory bluff-audit + permanent guard. The read-and-verify analyzer is itself anti-bluff (golden-good PASSes, golden-bad FAILs, §1.1).

(F) Vision analysis MANDATORY for every recording. After every recording, the agent MUST perform vision analysis: read the terminal output / screenshot frames / video content and verify (i) the feature ACTUALLY WORKS (real output, not errors, not empty), (ii) LLM responses are REAL (not simulated, not placeholder, not "for now"), (iii) all tests show PASS with captured evidence, (iv) no "TODO implement", "simulate", "for now", or "in production this would" patterns appear, and (v) the output demonstrates the feature working for the end user. Vision validation MUST produce a verdict (PASS/FAIL) with cited evidence path (§11.4.69). A recording without vision validation is a §11.4.158 violation — a video that was never read is NOT evidence.

Honest boundary (§11.4.6): reading + verifying the shown content proves the recorded run produced a genuine working result for the cases exercised — it does NOT prove zero remaining defects (§11.4.118) and does NOT replace §11.4.108 runtime-signature verification or §11.4.40 full-suite retest; it is the recorded-evidence layer every feature additionally crosses.

Classification: universal (§11.4.17) — the consuming project supplies its concrete capture mechanism, OCR/read tooling, HelixQA banks, and (optional) recording-path override per §11.4.35. Composes §11.4.2 / §11.4.5 / §11.4.25 / §11.4.27 / §11.4.52 / §11.4.69 / §11.4.83 / §11.4.85 / §11.4.107 / §11.4.108 / §11.4.117 / §11.4.118 / §11.4.128 / §11.4.137 / §11.4.138 / §11.4.153 / §11.4.154 / §11.4.155 / §1.1. Propagation gate `CM-COVENANT-114-158-PROPAGATION` (literal `11.4.158`) + recommended gates `CM-INTENSIVE-RECORDING-COVERAGE` + `CM-RECORDING-CONTENT-READ-VERIFIED` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.158. Non-compliance is a release blocker. No escape hatch — no `--skip-recording-coverage`, `--video-without-content-read`, `--happy-path-recording-suffices`, `--recording-path-anywhere`, `--unread-recording-OK`, `--skip-helixqa-read` flag.

§ . . — Mandatory window-specific video recording + vision validation mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-20): "All video evidence must be window-specific MP4 with mandatory vision validation, terminal cleanup, real results only."

Every feature test, validation, verification, challenge, and QA session that produces video evidence MUST comply with ALL of the following:

(A) Window-specific recording ONLY. Every video MUST record ONLY the target application window (Terminal pane, TUI app, browser tab, emulator frame), NEVER the whole desktop/monitor screen. Use window-specific capture mechanisms: macOS `screencapture -l<window_id>` (obtain window id via `osascript -e 'tell application "System Events" to get id of first window of process "<app>"'`), Linux `xdotool` + `ffmpeg -video_size WxH -f x11grab -i :0.0+X,Y`, or equivalent platform-specific window-targeted capture. Whole-desktop capture leaks operator-private content (§11.4.10), dilutes the §11.4.107 liveness/freeze oracle, and breaks the §11.4.137 OCR/ROI content oracle. Platform genuinely cannot capture below whole-screen => honest §11.4.3 SKIP + tracked migration item, never a whole-screen pass-off.

(B) MP4 format REQUIRED. Every recording MUST be saved as `.mp4` format (H.264 codec, `movflags +faststart`, `pix_fmt yuv420p`). `.cast` (asciinema) files are supplementary only — the primary evidence is the `.mp4` video. If using asciinema for terminal recording, auto-convert to `.mp4` via `agg` + `ffmpeg` immediately after capture per §11.4.154(C).

(C) Project-name prefix REQUIRED. Every recording filename MUST start with the project name in snake_case, derived from `HELIX_RELEASE_PREFIX` in `.env` (per §11.4.151) or the lowercased project root directory name (per §11.4.29). Format: `<project_name>-<feature>-<YYYYMMDD-HHMMSS>.mp4`. An unprefixed recording is a §11.4.159 violation (same severity class as §11.4.155).

(D) Mandatory vision validation. After EVERY recording, the agent MUST perform vision analysis: read the terminal output / screenshot frames / video content and verify: (i) the feature ACTUALLY WORKS (real output, not errors, not empty); (ii) LLM responses are REAL (not simulated, not placeholder, not "for now"); (iii) all tests show PASS with captured evidence; (iv) no "TODO implement", "simulate", "for now", or "in production this would" patterns appear; (v) the output demonstrates the feature working for the end user. Vision validation MUST produce a verdict (PASS/FAIL) with cited evidence path (§11.4.69). A recording without vision validation is a §11.4.159 violation — a video that was never read is NOT evidence.

(E) Terminal window cleanup. After each recording completes, the agent MUST dismiss/close ONLY the Terminal window used for that recording (using window-specific close: `osascript` with window id, `xdotool`, or equivalent). MUST NOT close Terminal windows belonging to other working processes. A recording that leaves orphan Terminal windows is a §11.4.159 violation.

(F) Real results ONLY. Every recording MUST show REAL working features — real API calls, real LLM responses, real test results. A recording showing errors, empty output, simulated responses, or placeholder content is a §11.4.159 violation and MUST trigger a fix -> retest -> re-record cycle before acceptance.

(G) Re-runnable evidence. Every recording MUST be reproducible: the command shown in the video MUST be re-runnable to produce the same results. Recordings of one-time manual interactions that cannot be automated are §11.4.159 violations.

(H) Fresh-corpus rotation. When a new recording run for a scope begins, the agent's own prior in-scope stale recordings at the recording path MUST be removed FIRST (per §11.4.154). Committed `docs/qa/<run-id>/` evidence is the durable record, NOT rotated.

(I) Content verification MANDATORY — not duration-based. The value of a recording is NOT its duration but its CONTENT. A recording MUST demonstrate the ACTUAL feature being used with REAL results. Before accepting ANY recording, the agent MUST verify:

- The expected output patterns ARE present in the recording (e.g., test PASS lines, API response data, feature-specific output)
- The feature ACTUALLY WORKS as demonstrated (not just "something ran")
- LLM responses are REAL content (not simulated, not placeholder, not empty)
- Every claim of "working" is backed by visible evidence in the recording

A 5-second recording that shows a feature working correctly is MORE valuable than a 60-second recording of empty terminal. Duration is NOT a proxy for quality.

(J) Expected-content specification REQUIRED. Before recording, the agent MUST specify what content SHOULD appear in the recording (expected patterns, expected test results, expected API responses). After recording, the agent MUST verify these patterns ARE present. If expected content is MISSING, the recording is REJECTED regardless of duration.

(K) Content-verification recording workflow. The mandated workflow for every recording:

1. SPECIFY expected content patterns (what SHOULD appear)
2. RECORD the feature execution
3. EXTRACT all text from the recording
4. VERIFY expected patterns are present
5. CHECK for simulated/placeholder content
6. ACCEPT only if ALL patterns found AND zero bluffs detected
7. REJECT and re-record if ANY pattern missing or bluff detected

(L) Root cause analysis REQUIRED for rejected recordings. When a recording is rejected (missing expected content, bluff detected, or empty capture), the agent MUST investigate WHY before re-recording. Per §11.4.102, determine the root cause (timing issue, wrong command, tool failure, etc.) and fix it. Simply re-recording without understanding WHY the first attempt failed is a §11.4.159 violation.

(M) Real-time monitoring RECOMMENDED. For complex features, use real-time monitoring that analyzes output DURING recording (not after). This catches issues immediately and allows corrective action before the recording completes.

Classification: universal (§11.4.17) — the consuming project supplies its concrete window-capture mechanism (per surface class), recording path, project-name prefix, and vision-validation tooling per §11.4.35. Composes §11.4.2 / §11.4.3 / §11.4.5 / §11.4.10 / §11.4.29 / §11.4.69 / §11.4.83 / §11.4.107 / §11.4.111 / §11.4.128 / §11.4.137 / §11.4.151 / §11.4.153 / §11.4.154 / §11.4.155 / §11.4.158 / §1.1. Propagation gate `CM-COVENANT-114-159-PROPAGATION` (literal `11.4.159`) + recommended gate `CM-WINDOW-VIDEO-VALIDATED` (every video is window-specific MP4 with vision-confirmed verdict + terminal cleanup + project-prefix) + paired §1.1 meta-test mutation (strip the literal -> propagation gate FAILs; a whole-screen recording where window-scoped is feasible, or a recording without vision validation -> `CM-WINDOW-VIDEO-VALIDATED` FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.159. Non-compliance is a release blocker. No escape hatch — no `--whole-screen-ok`, `--cast-only`, `--skip-vision-validation`, `--no-cleanup`, `--simulated-recording-ok`, `--unprefixed-recording` flag.

§ . . — Vision-verified recording + HelixQA bridge mandate (User mandate, - -)

Forensic anchor — direct user mandate (2026-06-21). Every video recording produced for feature/QA evidence under this Constitution MUST be processed through a vision/OCR pipeline that reads the on-screen content and confirms expected results BEFORE the recording is accepted as evidence. A recording that is accepted without automated read-the-screen verification is a §11.4 PASS-bluff at the evidence-integrity layer — "a video was produced" does not prove the video contains a genuine working result, and operator-side manual review does not scale to the intensive recording mandate (§11.4.158).

The bridge requirement. The recording system MUST provide a real-time or post-recording bridge that feeds captured frames/pixels to HelixQA's test infrastructure (or equivalent automated test framework) for content verification — automated read-the-screen verification against expected content patterns specified BEFORE recording (per §11.4.159(J)). The bridge MUST:

1. **Capture frames at ≤5 s intervals** during the recording window — sufficient to assert the feature's progression, not merely its start and end state.
2. **Run OCR/vision analysis against each frame** using a self-validated golden-good/golden-bad analyzer per §11.4.107(10) — the analyzer itself MUST be mutation-tested so it cannot bluff a PASS on empty/degraded content.
3. **Compare extracted text against the SPECIFY-phase expected patterns** — every pattern from the §11.4.159(J) expected-content specification MUST be checked.
4. **Produce a per-frame PASS/FAIL verdict with an evidence path to the frame** — raw frame image + OCR output + comparison result, linked from the verdict.
5. **Surface failures immediately** so the recording can be re-done per §11.4.159(L) root-cause analysis, not queued for batch review after the recording window closes.

Project-configured parameters (per §11.4.35). The frame-capture interval, OCR confidence floor, and pattern-matching thresholds are project-configured, calibrated on the project's own fixtures per §11.4.6 (no hardcoded industry defaults).

Honest boundary (§11.4.6). Vision verification confirms the feature produced expected on-screen output — it does NOT replace §11.4.5 captured-evidence quality analysis (audio RMS/XRUN, video freeze/jitter/obstruction census) nor §11.4.108 runtime-signature verification (the artifact's bytes landed and are active on a clean target). If the recording surface is FLAG_SECURE / DRM-protected / sink-blanked per §11.4.112, the test documents the gap and uses the §11.4.117 proxy oracle (accessibility cue / subdebug channel) instead — never a faked pixel-verified PASS.

Composition. §11.4.160 composes with §11.4.5 (captured-evidence quality — §11.4.160 adds the automated read-the-screen layer, does not replace quality metrics), §11.4.27 (HelixQA is the framework that runs the bridge), §11.4.69 (verdict with evidence path), §11.4.107(10) (self-validated analyzer), §11.4.112 (secure-surface gap documented), §11.4.117 (proxy oracle for non-introspectable surfaces),

§11.4.153 (per-feature video ledger), §11.4.158 (intensive recording coverage — §11.4.160 adds the verification bridge to every recording), §11.4.159(J) (SPECIFY-phase expected-content patterns are the bridge's input).

Gates. Propagation gate `CM-COVENANT-114-160-PROPAGATION` (literal `11.4.160`) + recommended gate `CM-VISION-VERIFIED-RECORDING-BRIDGE` (every recording has a self-validated OCR/vision pass against SPECIFY-phase patterns, with per-frame PASS/FAIL) + paired §1.1 meta-test mutation (strip the bridge/accept a recording without vision verification → gate FAILs; gate-code = separate work item).

Classification: universal (§11.4.17) — the consuming project supplies its concrete capture mechanism, OCR/vision tooling, capture interval, confidence floor, and HelixQA bridge integration per §11.4.35. No escape hatch — no `--skip-vision-verification`, `--manual-review-suffices`, `--no-bridge`, `--single-frame-suffices`, `--hardcoded-thresholds-OK`, `--unvalidated-analyzer-OK` flag exists.

2026 06 21 11 4 163

§ . . — Universal Media Validation & Verification Mandate (User mandate, - -)

Forensic anchor — direct user mandate (2026-06-21).

The mandate. Every recorded artifact (video MP4, audio WAV, screenshots PNG, asciinema cast, text output) produced by any project governed by this Constitution MUST pass through a MEDIA VALIDATION pipeline before being accepted as evidence. A recording that has NOT been validated by the pipeline is a §11.4 PASS-bluff at the evidence layer — it asserts the content is genuine and correct without having verified it.

(a) Content extraction. The pipeline MUST extract content from the media — OCR for video/screenshots (per §11.4.117 with a confidence floor per §11.4.107(12)), transcription for audio, text parsing for asciicast and text output. The extraction tooling is self-validated per §11.4.107(10) — a golden-good fixture MUST produce a PASS, a golden-bad fixture MUST produce a FAIL; an extractor that passes its golden-bad fixture is a bluff gate.

(b) Pattern matching. The extracted content MUST be compared against the SPECIFY-phase expected patterns per §11.4.159(J). Every specified pattern must be matched by at least one extracted line/frame; any specified pattern with no match yields a FAIL with an exact pinpoint record (which pattern, what the extracted content contained instead). The pattern set is the closed set of expected strings the test author declares before recording; an un-patterned PASS is inadmissible.

(c) Self-validated analyzer. The media validation pipeline as a whole (extraction + matching + verdict) MUST be mutation-tested with a golden-good/golden-bad fixture pair (§11.4.107(10)) — the golden-good MUST PASS, the golden-bad MUST FAIL. A pipeline that passes its golden-bad fixture is itself a §11.4 bluff gate; the pipeline MUST NOT be used for any evidence until its self-validation is restored.

(d) Structured verdict. The pipeline MUST produce a structured verdict for each artifact containing: PASS/FAIL, exact evidence path (file + frame/line/timestamp), matched patterns list (pattern → extracted text → frame/line), unmatched patterns list (pattern → expected → actual), analyzer version and calibration hash. The verdict is committed alongside the artifact under `docs/qa/<run-id>/` (§11.4.83).

(e) Exact pinpoint on FAIL. When the verdict is FAIL, the pipeline MUST provide pinpoint data — exactly which media artifact, which frame (video) or line (text/asciicast) or timestamp (audio), which OCR region or audio segment, which expected pattern, and what the extracted content contained instead. The pinpoint is the root-cause entry point for §11.4.102 systematic-debugging before re-recording per §11.4.159(L).

(f) Dual trigger mode. The pipeline MUST be triggerable both as a post-recording step (validate after the full recording completes, per §11.4.159(K)) AND as a real-time monitoring step that analyses frames/output during recording per §11.4.159(M) — real-time mode catches failures immediately, reducing re-record overhead.

(g) Paired §1.1 meta-test mutation. The media validator is itself subject to paired §1.1 mutation — a golden-bad fixture that the validator MUST reject, or the validator is a bluff gate. The mutation test creates a golden-bad fixture, feeds it to the validator, and asserts FAIL. If the validator returns PASS on the golden-bad fixture, the mutation test FAILs and the validator is inadmissible until fixed.

Honest boundary (§11.4.6). The media validation pipeline confirms the artifact contains the content it claims to contain and matches the intended patterns — it does NOT prove the feature is correct for the end user (that rests on §11.4.5

captured-evidence quality analysis, §11.4.107 liveness, and §11.4.108 runtime-signature), and it does NOT replace §11.4.40 full-suite retest. A pipeline that can be bypassed (a recording accepted without validation) is a §11.4.163 violation — the pipeline is mandatory for every recorded artifact, never optional.

Classification: universal (§11.4.17) — the consuming project supplies its concrete media extractors (OCR engine, transcription engine, text parser), golden-good/golden-bad fixture pair, expected-pattern declaration mechanism, and structured-verdict template per §11.4.35. Composes §11.4.107(10)/.112/.117/.159(J)/.159(K)/.159(M)/.83/.5/.69/.102/.134/§1.1. Propagation gate `CM-COVENANT-114-163-PROPAGATION` (literal `11.4.163`) + recommended gate `CM-MEDIA-VALIDATION-PIPELINE` (every recorded artifact crosses a self-validated media validation pipeline before acceptance; golden-bad fixture produces FAIL) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; bypass the pipeline and accept a raw unvalidated recording → `CM-MEDIA-VALIDATION-PIPELINE` FAILs; gate-code = separate work item).

§11.4.193 — Anti-blind-typing mandate: every UI interaction MUST be "seen" and "understood" via OCR/vision/screenshot proof, NEVER blind-typed (User mandate, 2026-07-13). Verbatim operator mandate: "No blind typing of any kind anywhere!!!" and "Login to apps and services MUST BE done with understanding and seeing the whole UI ALWAYS!!!" and "Add into the constitution that blind typing is STRICTLY FORBIDDEN, and instead everything MUST BE seen and understood in coordination with proper models and means we have incorporated in HelixQA!!!" and "No bluff of any kind anywhere!!!"

Blind typing — sending keystrokes, text, or input events to a UI surface WITHOUT first capturing AND understanding what is on screen (OCR / vision / uiautomator / accessibility-tree / screenshot) AND confirming the result after each interaction — is STRICTLY FORBIDDEN everywhere and always, for every agent, every tool, every automation, every platform, every context. Blind typing is LITERAL bluff: the agent claims "credentials entered" or "login completed" without visual proof the keystrokes landed on the intended fields, on the intended screen, producing the intended result. A credential typed into the wrong field, onto the wrong screen, or into a WebView that swallowed the input silently is a §11.4 PASS-bluff at the UI-interaction layer — the agent reported success while the end-user feature (logged-in app) is broken.

(1) SEE-BEFORE-YOU-TYPE. Before ANY input text, input keyevent, adb shell input, uiautomator gesture, or equivalent UI-driving command, the agent MUST capture the current screen state via screencap + OCR OR uiautomator dump OR accessibility-tree dump OR vision-model frame AND confirm the intended target element is visible AND focused. The captured screen state IS the evidence per §11.4.5 and §11.4.69 feature class touch_input.

(2) VERIFY-AFTER-YOU-TYPE. After EVERY interaction step, the agent MUST capture a NEW screen state and confirm the intended change occurred. A sequence of 5 input text commands with a single screenshot at the end is blind typing — each step needs its own verification.

(3) OCR/VISION ORACLE MANDATORY. The verification MUST use at least ONE of: Tesseract OCR on a screencap at 150+ DPI, uiautomator dump with grep for expected text content, HelixQA VisionEngine frame analysis per §11.4.159 and §11.4.160, or accessibility-tree dumpsys confirming element state.

(4) WEBVIEW SURFACES. When uiautomator returns near-empty hierarchy (common in streaming-app login WebViews), the agent MUST fall back to OCR via screencap then tesseract per §11.4.117. If even OCR is infeasible (secure surface captures black, DRM-blanked), the login is genuinely operator_attended only — SKIP-with-reason per §11.4.3, NEVER blind-type-and-claim-success.

(5) CREDENTIAL-SAFE OCR. When OCR output may contain credentials, the agent MUST redact credential characters from OCR output and log files per §11.4.10. Evidence paths cited via ab_pass_with_evidence; evidence file is the redacted screenshot or OCR text with credential characters replaced.

(6) ANTI-BLUFF. Every login/interaction PASS MUST cite a captured-evidence artefact path containing pre-interaction screen state, post-interaction screen state, and OCR/vision verification showing the expected change. A PASS without these artefacts is a §11.4 PASS-bluff regardless of exit code. Self-validated per §11.4.107(10): the OCR/vision oracle MUST ship a golden-good and golden-bad fixture pair.

Classification: universal (§11.4.17). Composes §11.4 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.117 / §11.4.159 / §11.4.160 / §11.4.10 / §11.4.52 / §11.4.3. Propagation gate CM-COVENANT-114-193-PROPAGATION (literal 11.4.193) +

recommended gate CM-NO-BLIND-TYPING + paired §1.1 mutation (strip OCR verification before an input text → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md §11.4.193.

Non-compliance is a release blocker regardless of context. No escape hatch — no blind-typing-ok, skip-ocr-verification, assume-input-landed, login-without-proof, type-without-seeing flag exists.

§ . . — Every guard/gate MUST assert the REAL condition: a false-positive refusal is a FAIL-bluff, a false-negative pass is a PASS-bluff (research-derived, - -)

Forensic anchor (FACT, 2026-07-15 — two independent instances in one cycle). (a) A host-budget guard used a bare `pgrep -f '<pattern>'` and REFUSED every worker spawn on a host with ZERO real builds, because a live worker's own command line merely QUOTED the pattern (the §11.4.196(D) / §12.12 carrier footgun). (b) A host-safety pre-flight refused a build on a "swap full" reading that did not reflect a real memory-pressure condition, and cleared only after the operator manually cycled swap. In BOTH cases a guard refused on a PROXY signal that was NOT the real condition — halting real work while reporting a condition that did not exist.

Every GUARD / GATE / PRE-FLIGHT CHECK (host-safety, resource-budget, readiness, lock, capability, availability) MUST assert the REAL condition it CLAIMS to assert, resolved from the AUTHORITATIVE source — the process's real `/proc/<pid>/cmdline`, the real cgroup / meminfo pressure metric, the real lock-holder liveness (`kill -0`), the real device identity — NEVER from a proxy signal that something which is NOT the condition can satisfy.

(1) A FALSE-POSITIVE REFUSAL IS A FAIL-BLUFF (§11.4.1). A guard that BLOCKS work when the condition is ABSENT is exactly as forbidden as a gate that PASSES while a defect is present: it halts real work, it teaches operators and agents to bypass guards (the worst possible second-order effect), and it hides the true state behind a confident false claim.

(2) A FALSE-NEGATIVE PASS IS A PASS-BLUFF (§11.4). A guard that does NOT fire when the condition IS present is the classic bluff — the guard exists precisely to catch that state.

(3) SELF-VALIDATED (§11.4.107(10)). Every guard MUST ship a golden-TRUE fixture (the real condition PRESENT → the guard FIRES) AND a golden-FALSE fixture that INCLUDES the known CARRIER / decoy case (the condition ABSENT but the proxy signal present → the guard MUST NOT fire), both wired into the meta-test. A guard that fires on its golden-FALSE fixture is ITSELF the defect, and shipping a guard without both fixtures is a §11.4 bluff at the guard layer.

(4) CONSERVATIVE-SAFE DEFAULT ON AN UNRESOLVABLE SIGNAL (§11.4.101 / §12). When the authoritative source genuinely cannot be read, the guard takes the SAFE, reversible default (refuse for a destructive / high-blast-radius action; the safe side for a host- or hardware-safety guard) and SAYS SO HONESTLY. A conservative refusal WITH an honest "could not resolve the condition" reason is NOT a false positive; a refusal that CLAIMS a condition it never verified IS one (§11.4.6).

(5) EVERY REFUSAL REPORTS ITS RESOLVED EVIDENCE. A guard refusal MUST print the resolved evidence that triggered it (the real pid + cmdline, the real metric + threshold, the real lock holder + liveness result) so the refusal is auditable and a false positive is diagnosable in one step rather than one session.

(6) SCOPE — EVERY LOAD-BEARING MEASUREMENT, NOT ONLY GUARDS (extension, research-derived, 2026-07-17). Clauses (1)-(5) bind guards / gates / pre-flights — decision seams that REFUSE or ALLOW — and their failure taxonomy is refusal-shaped. But the same defect kills an ad-hoc measurement that refuses nothing: a `grep`, a count, a parse, an exit-code check, a string scan, a burn-down metric, an artifact inspection. These are the instruments an agent forms BELIEFS from, and a false belief propagates into every downstream decision without ever passing a guard. Therefore clauses (6)-(8) bind **every measurement whose result is load-bearing** — i.e. any observation an agent, gate, report, or metric relies on to assert a fact. Their taxonomy is MEASUREMENT-shaped: **a FALSE-NULL** (a zero / empty / no-match / "absent" read as evidence of absence when the instrument simply could not see) and **a FALSE-MATCH** (a hit on

something that MENTIONS the thing rather than IS it) are both §11.4/§11.4.1 bluffs at the measurement layer — the null one is the more dangerous, because a broken instrument and a genuinely-clean artifact return the identical, confident, quiet zero.

(7) CARRIER-VS-THING + THE CONTROL NEEDLE. (a) MATCH STRUCTURE, NOT SUBSTRING. A measurement MUST distinguish the THING from a CARRIER that merely MENTIONS it. Forensic FACTs, all genericised, all real, all in ONE cycle of ONE project: a removal's completeness was measured by counting references to the removed component — and the count matched the AUDIT-TRAIL COMMENTS documenting the removal, so a completed removal read as a failed one (the §11.4.122 audit trail NAMES what it removed; naming is not using); a "does the artifact contain X" scan matched the governance text DOCUMENTING X; and this anchor's own siblings had already booked the same shape twice — §11.4.196(D)'s process-carrier (a process quoting the pattern is not a process being the thing) and §12.12's self-match (the scanner's own command line contains the pattern it scans for). **A defect already anchored twice and still recurring is the argument for the general rule, not against it.** The instance that decides it: a mutation-residue scan run over a governance repository flagged the very anchor that DEFINES the mutation markers, because that anchor quotes them in its rule text — the enforcement mechanism false-positived on its own law. **(b) A NULL IS NOT EVIDENCE UNTIL A CONTROL NEEDLE PROVES THE INSTRUMENT CAN SEE.** Before a zero / empty / no-match / "absent" result may be reported as a real absence, the SAME instrument, over the SAME path, against the SAME artifact, MUST be shown to return a non-null for a **control needle** — a string/property KNOWN to be present. Needle NOT found ⇒ the instrument is blind and the zero says NOTHING about the artifact (§11.4.6: report the blindness, never the absence). **The needle MUST be expressed through the SAME load-bearing query features as the query it certifies** — same dialect constructs (alternation / character class / quantifier / escape), same anchoring, same quoting, same encoding path. A bare-literal needle certifies ONLY the layers a bare literal crosses; it does NOT certify a query whose meaning depends on a feature the needle never used. **This bound is not theoretical — the (7)(c) dialect FACT refutes the naive form directly:** a query intending alternation but written with an escape that the dialect treats as a LITERAL pipe searches a string nobody wrote and returns zero; a plain-literal needle sails through the identical tool + path, returns non-null, and would "certify" that zero — while the query is still blind. A needle carrying the SAME alternation (one arm known-present) dies with the query and exposes it. Hence the

earned inference is bounded: **needle found ⇒ the path can see the needle's QUERY CLASS ⇒ a zero from a query of that same class is evidence**; a zero from a query using features the needle did not exercise is NOT yet evidence, and needs a needle of its own class. This generalises §11.4.115(F)'s validate-the-detector principle (a guard never observed FAILing on the genuinely-broken artifact is unvalidated instrumentation and mints no verdicts) from guards minting verdicts to every measurement, and it is PER-QUERY, against the artifact in hand — **golden fixtures cannot substitute** (§11.4.146(D3): "goldens are necessary, not sufficient"), because an instrument can pass every fixture on the authoring host and still be blind on the real path. **(c) THE PATH IS PART OF THE INSTRUMENT.** The instrument is not the tool, it is the tool PLUS every layer the query crosses, and the load-bearing failures live in the layers: a quoted pattern whose escapes are consumed by an intervening/wrapper shell, so the far side searches a LITERAL where alternation was intended and matches nothing; `\|` under an extended-regex dialect where it is a LITERAL pipe, not alternation, silently searching for a string nobody wrote; a pipeline (`check | head` , `check | xargs`) whose exit status is the LAST stage's, so a checker's verdict is discarded and its "clean ✓" is the pager's success, repeatable indefinitely without ever checking anything; a pattern anchored to line-start that misses an indented or colour-escaped line, reading a present verdict as no-verdict; a binary/encoding-sensitive extractor that splits a multi-byte character and reports a present string as absent. Each of these returns a CLEAN, CONFIDENT, WRONG answer — none crashes, so §11.4.1's script-bug clause (which covers failures that crash into a FAIL) never fires, and §11.4.67's parse-check (which is parse-time only) cannot see it. Hence (7)(b) is not optional diligence: a class-matched control needle is the only mechanism that catches these defects, because it is the only one that exercises the whole path with the query's own features. **Honest boundary (§11.4.6):** the needle proves the path can SEE its own query class — it does NOT prove the query is semantically the RIGHT question to ask (a correct-and-visible query for the wrong property still misleads); that remains (8)'s and §11.4.201(1)'s concern.

(8) A METRIC MUST BE VALIDATED AGAINST THE DEFINITION OF DONE. A progress / burn-down / coverage / ratchet metric is a load-bearing measurement, and a metric that is ANTI-CORRELATED with done is not a misread condition — it is the WRONG condition, read correctly. Forensic FACT (genericised): a removal's burn-down counted references to the removed component and was tracked toward zero — but the CORRECT end-state ADDS references, because a retired gate

must carry a mandated rationale naming what was removed and why, and the inverse "stays-removed" assertion must NAME the removed thing to assert its absence; the number therefore RISES as the work succeeds, the target is unreachable, and chasing it to zero would DELETE the very rationale and assertions the constitution mandates. Therefore: before a metric may gate, rank, or report work, it MUST be checked against the definition of done — construct the correct end-state and evaluate the metric there; **if the correct end-state does not move the metric to the target, the metric is INVALID and MUST NOT gate work** (a metric whose target the correct outcome cannot reach will, if enforced, drive work AWAY from correctness — the strongest possible form of §11.4.201's proxy-signal defect, because here the proxy is not merely satisfiable by a non-condition, it is REFUTED by the condition). This is the (7)(a) carrier defect relocated from a guard to a metric: the audit-trail rationale and the stays-removed assertion are CARRIERS, and the burn-down was a substring-counter that carriers satisfy. **Honest boundary + standing obligation (§11.4.6):** this constitution already ships at least one gating metric of exactly this class — §11.4.135's adoption ratchet, a monotone-decrease burn-down that BLOCKS the release tag (§11.4.135's seam; NOT the build) — and its validation against the definition of done is OWED, not hereby claimed; naming it is the §11.4.118 discovery-pressure obligation. That validation **MUST be tracked** as a §11.4.197 work item by every consuming project that operates a ratchet (stated as the obligation it is — this anchor does NOT assert that any particular project has already filed it; asserting an unverified tracker state would be the §11.4.6 violation this very clause exists to forbid). **Interim precedence (§11.4.6 — stated, never left for a consumer to guess):** until that validation lands, §11.4.135(5)'s ratchet KEEPS GATING — clause (8) does NOT retroactively disarm it, because an unvalidated ratchet that blocks a release is a bounded, visible, operator-resolvable stop, whereas disarming the only adoption mechanism would silently re-open the unguarded-backlog hole §11.4.135 exists to close (the §11.4.101 reversible-safe choice). Clause (8) binds (a) every NEW metric from this anchor forward — none may gate until validated against the definition of done — and (b) the ratchet's own validation as owed work, NOT as a licence to ignore it. A consuming project MUST NOT cite clause (8) to switch its ratchet off.

GENERALISES §11.4.196(D) (resource-detection-by-real-identity — from the alias / build guard to EVERY guard) and §11.4.180 (holder-liveness PROVEN via `kill -0` before reaping — to every guard's condition). Classification: universal (§11.4.17) — the consuming project supplies its concrete guards, authoritative

sources, and carrier/decoy fixtures per §11.4.35. Composes §11.4.1 / §11.4.6 / §11.4.101 / §11.4.107(10) / §11.4.109 / §11.4.111 / §11.4.180 / §11.4.196(D) / §12 / §12.6 / §12.8 / §12.12. Propagation gate `CM-COVENANT-114-201-PROPAGATION` (literal `11.4.201`) + recommended gate `CM-GUARD-ASSERTS-REAL-CONDITION` (every guard resolves its condition from the authoritative source, ships golden-TRUE + golden-FALSE-with-carrier fixtures wired into the meta-test, and prints its resolved evidence on refusal) + paired §1.1 mutation (replace a guard's authoritative resolution with a bare substring / proxy match → its golden-FALSE carrier fixture makes the gate FAIL; strip the literal → the propagation gate FAILs; gate-code = separate work item).

Gate contract for clauses (6)-(8) (extension, 2026-07-17). `CM-GUARD-ASSERTS-REAL-CONDITION` is EXTENDED beyond its guard-fixture invariants to cover the measurement layer, and a metric-specific sibling is added — because a clause landing with no gate + no paired mutation is itself the §1.1 bluff-gate pattern this anchor exists to forbid: **(i)** every load-bearing NULL/ZERO/absence result reported as a finding cites a control-needle result of the SAME query class, captured alongside it (a reported absence with no class-matched needle evidence → FAIL); **(ii)** the needle's query class is asserted to match the certified query's load-bearing features (a bare-literal needle certifying an alternation/anchored/encoded query → FAIL — the finding-1 defect this clause was corrected for); **(iii)** recommended gate `CM-METRIC-VALIDATED-AGAINST-DONE` — every metric that gates, ranks, or reports work carries a recorded definition-of-done evaluation showing the correct end-state reaches the target (a gating metric with no such record, or one whose recorded correct-end-state does NOT reach target, → FAIL). Paired §1.1 mutations: swap a class-matched needle for a bare literal on a dialect-dependent query → (ii) FAILs; strip the needle entirely from a reported absence → (i) FAILs; register a metric whose correct end-state moves it AWAY from target → (iii) FAILs. Gate-code = a separate work item; this is the contract, not a claim that the code has shipped (§11.4.6).

(9) FIELD-IDENTITY + UNITS + SEMANTICS — capacity is not occupancy (extension, research-derived, 2026-07-23). Forensic FACT (genericised, 2026-07-22): a load-bearing latency claim — "a standing ~1.4-second send queue" — was minted by reading a socket diagnostic's WINDOW-CAPACITY field (`rcv_wnd` 181120 bytes, the ceiling the peer ADVERTISES) as if it were QUEUE OCCUPANCY; the real observed Send-Q was a few hundred bytes (in-flight-sized, `app_limited`), three orders of magnitude smaller. The misread propagated into

THREE parallel agents' briefs and into operator reports before anyone re-read the raw field; two INDEPENDENT re-derivations from the raw samples then converged on the correction, and the false figure had to be publicly withdrawn from a changelog. Therefore: before ANY metric read from a summary / diagnostic line / dashboard cell may gate a decision, rank work, or propagate into a brief / report / tracker, its FIELD IDENTITY (which field, exactly, in the authoritative raw output), its UNITS, and its SEMANTIC CLASS — capacity/limit vs occupancy/backlog vs rate vs cumulative counter vs high-water mark — MUST be verified against the authoritative source's own field definition (§11.4.99 latest-source where the tool's documentation defines it). CAPACITY≠OCCUPANCY is the named canonical instance; limit-vs-current, cumulative-vs-delta, and advertised-vs-measured are the same defect. This is the clause-(6) FALSE-MATCH at the metric layer — the instrument matched A number, not THE quantity — and a downstream consumer of an unverified metric INHERITS the bluff (§11.4.6: re-derive from the raw source before re-publishing; a figure carried between agents/briefs without its field-identity verification is a rumour with units).

(10) OBSERVER DECONTAMINATION — the instrument's own footprint is identified, excluded, and the decontaminated instrument re-proven seeing (extension, research-derived, 2026-07-23). Forensic FACT (genericised, 2026-07-22): a socket census read TIME-WAIT counts of 68–80 and nearly minted a leak finding — after excluding the SAMPLER'S OWN probe ports the real count was 4–5 (the census was ~16× observer-induced); the control needle proving the decontaminated instrument still SAW real state: the same scan did detect a genuine step-change (+264 MB RSS, 1→93 sockets) in a monitored process. Every measurement whose instrument CONTRIBUTES to the measured population (probe connections in a socket census, sampler CPU/IO in a load profile, probe processes in a process census, trace overhead in a latency measurement — the §12.12 self-match generalised from kill-lists to counts) MUST identify and EXCLUDE its own footprint before reporting, AND MUST prove via a (7)(b)-class control needle that the decontaminated instrument still sees genuine signal (an over-broad exclusion is a self-inflicted clause-(6) false-null). Composes §11.4.128(1) (observer-effect budget — the recorder must not perturb the recorded; clause (10) adds the measurement-side half: where the footprint is unavoidable it is measured and SUBTRACTED, never reported as the system's own behaviour) and §11.4.119 (single-resource-owner — a measurement arm racing a concurrent driver

of the same resource produces cross-contaminated evidence; partition or serialize FIRST, as measured work this cycle did by refusing to touch a sibling stream's in-flight file region).

(11) ARTIFACT-USABILITY, NOT PREREQUISITE-PRESENCE (extension, research-derived, 2026-07-23). Forensic FACT (genericised, 2026-07-22): an install/verify pipeline keyed its "can this host run the tool?" verdict on the presence of a BUILD prerequisite (a `command -v <compiler>` probe) — and turned a host holding a WORKING prebuilt binary into a false "cannot launch" FAIL, exit 1, in one transcript that showed BOTH "[ok] ... verified" AND "[FAILED] ... cannot launch" for the same artifact; the independent §11.4.142 review graded it BLOCKING (it breaks every upgrade on hosts that install from prebuilts). A readiness / install / health / build gate whose CLAIM is about an ARTIFACT ("the tool works here") MUST probe the ARTIFACT ITSELF — execute it through its real invocation path and assert its observable behaviour (§11.4.224(A)'s real-invocation-path discipline; §11.4.200's verify-the-target sibling at the readiness seam) — NEVER the presence of a prerequisite for ONE of several ways of producing it (a compiler, an SDK, a package manager). The proxy fails in BOTH directions at once: prerequisite-present ≠ artifact-usable (a toolchain with no artifact) AND prerequisite-absent ≠ artifact-unusable (a prebuilt with no toolchain) — clause (1) and clause (2) in a single check.

(12) SHELL-INSTRUMENT FOOTGUN CHECKLIST — a standing (7)(c) reference that shell-touching reviews MUST consult (extension, research-derived, 2026-07-23). The measured shell-instrument footgun classes are catalogued in the standing reference `docs/guides/shell_instrument_footguns.md` (inherited by reference per §11.4.28/§11.4.177, never copied per project). Among the founding, DEMONSTRATED classes: `timeout` / `nice` / `env`-class EXEC WRAPPERS cannot run shell functions/aliases (rc=127 "command not found" while the function is healthy — a clause-(1) false refusal; demonstrated 2026-07-23 under control conditions); `pgrep -f` / `pkill -f` matching the PROBING script's own command line (the §11.4.196(D)/§12.12 carrier — re-demonstrated 2026-07-23: a unique needle matched exactly ONE process, the probe's own harness); structural-delimiter text extractors (a column-0-`}`-terminated awk function extractor) that TRUNCATE the extracted unit when its body legitimately contains the delimiter (heredoc/string content) and hand the test a PARTIAL artifact — a FALSE PASS when the truncated tail carried the code under test (demonstrated 2026-07-23: the tail marker vanished from the

extraction); and xtrace (`bash -x`) BLINDED when the code under test redirects the shell's stderr (`exec ... 2>/dev/null` silenced the trace mid-stream in the real 2026-07-22 incident — countermeasure: route the trace via `BASH_XTRACEFD` to a dedicated fd the code under test does not own). Every §11.4.142/§11.4.209 code review of a shell-based test, gate, guard, or measurement MUST consult the checklist, and every zero/absence/PASS a shell instrument produces remains bound by (7)(b)'s class-matched control needle. The checklist is a LIVING document: a newly-measured footgun class is APPENDED with its captured demonstration (§11.4.5) in the same change-window, never carried as tribal memory (§11.4.215 — the binding reference lives tracked, in-repo).

Gate contract for clauses (9)-(12) (extension, 2026-07-23). `CM-GUARD-ASSERTS-REAL-CONDITION` is EXTENDED: **(iv)** every load-bearing metric cited in a gating / ranking / report context carries its verified field-identity + units + semantic class (capacity / occupancy / rate / cumulative / high-water) traced to the authoritative source definition — an unverified-semantics metric gating a decision → FAIL; **(v)** every census/profile whose instrument contributes to the measured population records its observer-exclusion AND a post-exclusion control-needle result — an undecontaminated census minting a finding → FAIL; **(vi)** every artifact-readiness verdict cites a probe of the artifact's own invocation path — a prerequisite-presence-only readiness verdict → FAIL; **(vii)** every review record touching a shell-based instrument cites the §11.4.201(12) checklist consultation. Paired §1.1 mutations: relabel a capacity field as occupancy in a gating metric → (iv) FAILs; strip the observer-exclusion from a census fixture → (v) FAILs; swap the artifact probe for a bare `command -v <compiler>` while the fixture host carries a working prebuilt → (vi) FAILs. Gate-code = a separate work item; this is the contract, not a claim that the code has shipped (§11.4.6).

Canonical authority: constitution submodule `Constitution.md` §11.4.201.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--proxy-signal-OK` , `--substring-match-guard` , `--false-positive-refusal-acceptable` , `--unvalidated-guard` , `--refuse-without-evidence` , `--zero-proves-absence` , `--skip-control-needle` , `--substring-is-good-enough` , `--carrier-match-OK` , `--trust-the-null` , `--metric-without-done-check` , `--gate-on-unvalidated-metric` , `--capacity-is-occupancy-OK` , `--`

`skip-field-identity-check`, `--observer-footprint-included-OK`, `--prerequisite-presence-proves-artifact`, `--skip-shell-footgun-checklist` flag.

§ . . — Evidence-class-at-closure + standing detection pressure: machinery presence does NOT predict whether a fix holds — the EVIDENCE CLASS at closure (runtime vs source) under real DETECTION PRESSURE does; prose does not bind, seams do (research-derived, - -)

Forensic anchor (genericised — the 2026-07-22 reopen/first-touch root-cause forensics; operator mandate 2026-07-22, verbatim: "do deep research on how we have for very long time so much reopened workable items!? Why the quality of delivered releases was so poor mostly? We mostly detect the same major problems on first touch when new releases are available, and this happens over and over again!"). The single strongest empirical result across four independent evidence lines (internal forensics; a 76-source external literature pass; one cross-project corpus WITHOUT tracking machinery; one corpus WITH exemplary machinery): **every fix whose done-claim was preceded by an on-target RED→GREEN polarity flip that held recorded ZERO reopens; every fix confirmed only at source-green bounced.** Cross-corpus: the runtime-evidence-closure corpus recorded median attempts-to-stick = 1 and a ~0.3% corrected reopen rate (its own analysis marks that a FLOOR under weak detection pressure), while the source-green corpus recorded 52% — itself a proven undercount; the healthy corpus's ONLY bouncing fix family was closed by a wrong-layer echo-assertion (a unit test asserting a Noop echo — the discriminator's predicted loser, sign right); 69 commits in a two-month window moved claims at "PENDING-BUILD"; the last audited release entered manual QA with **1 of 25** in-scope items validated at the user-visible layer — the operator was the FIRST user-layer oracle most claims had ever met, so first-touch rediscovery of the same majors was the deterministic output of the pipeline shape, not bad luck; a pixel-layer defect was "verified" by a file-existence check plus five greps; and the corpus WITHOUT machinery shipped a "fully working" release hours before a total field brick with its whole hermetic (wrong-layer) suite green. The law this evidence selects: **machinery presence (trackers, guards, gates, anchors) is signal and custody,**

NOT prevention; what predicts whether a fix holds is the EVIDENCE CLASS AT CLOSURE (runtime vs source) under real DETECTION PRESSURE. Prose does not bind — seams do. Coverage has a blind complement at any maturity.

The mandate (ALL hold; the STRICT FORMALIZATION of §11.4.115(F)'s evidence-class clause + §11.4.146(D3)'s custody chain + §11.4.108's four layers — it supplies the closed taxonomy, rank floor, machine fields, and freshness contract those anchors did not state):

(1) Closed evidence-class ranking with a per-defect-layer floor, enforced at every closure seam. Defect layers form the closed set {user-visible, runtime, artifact, source} (§11.4.108); evidence classes form the closed RANKED set {runtime > artifact > source}. Every terminal or ready-class status write carries evidence whose class meets its defect layer's floor: a user-visible or runtime-layer defect can NEVER close on artifact- or source-class evidence; source-on-source stays legal (the §11.4.201(1) false-refusal guard — legitimate source-layer work is never refused).

(2) Class is proven by MACHINE FIELDS, never by a label. `runtime` requires a target fingerprint read FROM the target at run time (§11.4.115(F)) plus a runtime observable; `artifact` requires the artifact path plus its content hash; `source` requires the source ref. A class label without its fields is SHAPE-INCOMPLETE and refused; unreadable evidence is BLIND, never accepted.

(3) The ANTI-ECHO rule. A "runtime observable" that IS a grep/source-scan transcript is WRONG-LAYER and refused — a grep can never observe a pixel; the wrong-layer oracle dies by construction (the pixel-defect-by-five-greps closure and the Noop-echo family both cross this seam and are refused at it).

(4) Per-status evidence preconditions. Every status in the §11.4.15/§11.4.33 lifecycle carries a CHECKED evidence precondition — not only the terminal write. Forensic FACT: a penultimate "ready"-class status functioned as design-complete-but-UNVERIFIED and parked 79 items — 42% of the done-claiming population; the operator sampling "done" items was sampling a set dominated by never-verified work without either party being wrong about their own definition (semantic drift between claim-maker and claim-consumer). A status that sounds like progress but carries no checked precondition becomes the parking lot for unverified work.

(5) Fix-kind honesty. Every defect closure classifies its fix-kind; a STATE-ONLY repair (restart / cache-clear / re-deploy / manual re-run) or a mitigation CANNOT close a defect item as fixed — it is recorded as a mitigation and the item stays open or is honestly reclassified (composes §11.4.102's Iron Law + §11.4.146; forensic FACT: a sibling corpus's recurrence chains — 11 chains, median 3 and max 7 attempts per defect — were dominated by state-only "fixes" that closed and bounced).

(6) STANDING DETECTION PRESSURE. Detection pressure is a CAUSAL INPUT to every measured quality rate — a reopen/defect/quality figure reported without its detection-pressure regime is incomparable and misleading (§11.4.6): the measured 52%-vs-~0.3% cross-corpus split is explained by pressure + closure class, not machinery, and the healthy corpus's one genuine recurrence surfaced at 24 days — the next SWEEP, not the next use ("we only see what we test", §11.4.118). Therefore: every registered, topology-present guard carries a FRESHNESS CONTRACT — a verdict on the CURRENT artifact fingerprint, no older than a declared staleness budget; the never-executed / stale-fingerprint / over-budget set feeds a STANDING, risk-ordered re-run queue (most-reopened-first per §11.4.189, then stalest-first) that the zero-idle loop (§11.4.94/§11.4.97) drains as regular development-cycle work; the §11.4.135 release seam remains the blocking BACKSTOP, never the only executor. Forensic FACT: two standing guards, finally executed during the forensics, immediately emitted the FAILs that had been latent the whole time — one reproduced the operator's loudest defect 3/3; registration had been treated as coverage while execution was nobody's job. Topology-absent guards are enumerated, never queued (no false pressure, §11.4.201(1)); an empty registry is BLIND, never "all fresh"; staleness budgets are consumer data (§11.4.35) bounded by the alarm-fatigue trade-off — the queue is a work FEED, only the release seam blocks.

Regular-cycle + review + pull binding (operator mandate 2026-07-23). This anchor applies during EVERY regular development cycle (the freshness queue is standing §11.4.94 work, and every closure crosses seam (1)); it is CHECKED through every code-review per §11.4.194(6); and on every constitution fetch/pull the §11.4.32 post-pull validation sweep + the §11.4.164 auto-propagation hook analyze the new additions and run their gates against the post-pull tree, with violations tracked (§11.4.15) and cleared — no parallel mechanism is invented.

Honest boundary (§11.4.6). (i) The discriminator is a measured correlation with a stated mechanism and zero counter-cases in the record — NOT a randomized trial (guarded items had less operator exposure; the clean exemplars are few; the confound is carried, not hidden). (ii) Fabricated machine fields are not fully closed here — provenance stays with §11.4.115(F) harness-written verdicts + the §11.4.135 candidate-fingerprint join, where an invented fingerprint fails to match the real candidate and reads as absence; layered, not eliminated. (iii) Oracle calibration stays §11.4.107(10) — runtime-class evidence from a mis-calibrated analyzer still passes this seam; goldens are necessary, not sufficient. (iv) The defect-layer taxonomy is authored data — mislabelling a user-visible defect as source-layer lowers its floor; the §11.4.194(6)(a) review class is the detective counter (cheap to spot: the item's own description names user symptoms). (v) Re-running a weak guard often proves nothing — oracle strength remains §1.1/§11.4.115(F)'s job, and orthogonal discovery (§11.4.85 chaos, §11.4.118 discovery pressure) is a different feed this queue does not generate.

Reference mechanisms (proven, NOT wired — §11.4.6/§11.4.205). Runnable seam mechanisms with RED-first POCs, golden-good/golden-bad/negative-control fixtures (§11.4.107(10)), live-data runs, and stated boundaries exist in this repository's quality-solutions research corpus (`docs/research/quality/solutions/` — SOL-01 status-custody, SOL-02 verdict-coverage, SOL-04 evidence-class, SOL-09 detection-pressure; 65/65 GREEN across the ten-POC set on a deterministic second iteration). Per §11.4.205 they carry NO force until their seams are wired by separate tracked work items (§11.4.197); §11.4.227(A)'s ledger is the instrument that keeps that wiring debt visible.

Classification: universal (§11.4.17) — no hardware/vendor/project literal; the consuming project supplies its defect-layer bindings, evidence-store paths, verdict registries, and staleness budgets as DATA per §11.4.35. Composes §11.4.1 / §11.4.5 / §11.4.6 / §11.4.15 / §11.4.33 / §11.4.69 / §11.4.94 / §11.4.97 / §11.4.102 / §11.4.107(10) / §11.4.108 / §11.4.115(F) / §11.4.118 / §11.4.123 / §11.4.132 / §11.4.135 / §11.4.146(D3) / §11.4.185 / §11.4.189 / §11.4.194(6) / §11.4.201 / §11.4.205 / §11.4.214. Propagation gate `CM-COVENANT-114-226-PROPAGATION` (literal `11.4.226`) + recommended gates `CM-EVIDENCE-CLASS-AT-CLOSURE` (every terminal/ready-class status write carries class-matched, machine-field-proven evidence meeting its defect layer's floor; a grep-transcript "runtime observable" → FAIL; a state-only repair closing a defect as fixed → FAIL) and `CM-GUARD-FRESHNESS-SCHEDULER` (every registered, topology-present guard is fresh-

on-the-current-fingerprint or present in the visible, risk-ordered re-run queue; an empty registry → BLIND, never "all fresh") + paired §1.1 mutations (close a runtime-layer item on source-class evidence → the closure gate FAILs; attach a runtime label with no machine fields → FAILs; let a registered guard sit past its staleness budget with no queue entry → the scheduler gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6).

Canonical authority: constitution submodule `Constitution.md` §11.4.226.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--source-green-closes-runtime-defect`, `--label-is-class`, `--echo-observable-OK`, `--registration-is-coverage`, `--rate-without-pressure-regime`, `--mitigation-closes-defect`, `--skip-freshness-queue` flag exists.

§ . . — Machine-created evidence at every gate: every claim of "works" / "passes" / "verified" cites a captured, machine-derived, machine-verifiable evidence artifact produced by the gate — no operator eyeballing, no narrative-only PASS (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, - -)

Verbatim operator mandate (2026-08-15, Point 8): *"There MUST BE no bluff of any kind and in any form anywhere! Everything created MUST BE fully covered with confirmation, validation and verification gates which will assert machine created evidence so there is no prediction or guessing about the success and production ready state of the System!"*

Compact summary: every claim by the System that something WORKS / PASSES / IS VERIFIED / IS PRODUCTION-READY MUST be backed by MACHINE-CREATED, MACHINE-VERIFIABLE evidence — a file / verdict / measurement produced by the gate itself, cited by path + content-hash + timestamp, with a schema an independent verifier can parse (§11.4.226 evidence-class-at-closure applied to EVERY gate, not only closure seams); operator eyeballing, human narrative, "I checked it works", "looks fine", "no error was reported", and every other prediction / guess / assumption about success is FORBIDDEN at the evidence layer (§11.4/§11.4.1/§11.4.6). §11.4.262 STRENGTHENS the §11.4 anti-bluff covenant preamble from "captured evidence"

to MACHINE-CREATED evidence (a captured screenshot an operator interpreted is captured but not machine-verified — §11.4.107(10) golden-good/golden-bad self-validating analyzer applies UNIVERSALLY, not only to AV playback), and BINDS §11.4.5/§11.4.69/§11.4.107/§11.4.108/§11.4.115(F)/§11.4.201/§11.4.226 into one universal machine-evidence-at-every-gate discipline. Prediction, guessing, and assumption about success are the exact anti-patterns this anchor forbids.

(A) THREE-GATE COVERAGE. Every work product passes THREE gate classes each producing machine-created evidence: (1) CONFIRMATION — the change lands correctly (source review, static analysis, format/lint, dependency clash check per §11.4.110); (2) VALIDATION — the change behaves as designed at each of the §11.4.108 four layers (SOURCE / ARTIFACT / RUNTIME / USER-VISIBLE), each producing its own machine-created verdict + evidence artifact; (3) VERIFICATION — the change satisfies its acceptance contract from the user's / operator's / system's point of view (§11.4.185 manual-QA is the human sufficiency gate; §11.4.236 QA-deploy-readiness is the machine sufficiency gate — each with its own captured evidence).

(B) MACHINE-CREATED — the four properties. Every evidence artifact is (i) PRODUCED BY THE GATE, not authored by the tester (a screenshot the harness took, not one the operator narrated); (ii) MACHINE-VERIFIABLE — a schema-parseable file (JSON verdict, JSONL event stream, structured log, image with an OCR/vision oracle per §11.4.107(11)/§11.4.117, audio with RMS/loudness analysis per §11.4.107(11)) that an INDEPENDENT verifier can re-check without asking the tester; (iii) CONTENT-ADDRESSED — cited by path + sha256 + timestamp (§11.4.207 content-addressed Merkle) so tampering is detectable; (iv) SEMANTIC-CLASS-MATCHED — the evidence class matches the claim layer (§11.4.226 wrong-layer echo forbidden — a source grep transcript can never satisfy a "pixel renders green" claim).

(C) NO NARRATIVE-ONLY PASS. A PASS whose evidence is a paragraph of natural language authored by an operator or a subagent ("I ran the tests and they all passed", "looks good to me", "manually confirmed") is a §11.4.262 violation — the narrative may accompany the machine evidence but MUST NEVER substitute for it. §11.4.185 manual-QA remains a human-sufficiency gate whose OWN evidence is machine-created (a QA session recording per §11.4.128, a QA checklist file with per-item PASS/FAIL + evidence-path per §11.4.185, a §11.4.238 escape-log if the human found something automation missed).

(D) SELF-VALIDATED EVIDENCE ANALYZERS. Every evidence analyzer (the parser that reads the machine-created artifact and emits a verdict) is itself golden-good + golden-bad + negative-control validated per §11.4.107(10); an analyzer that PASSes its golden-bad fixture is the bluff and a release-blocker (the §11.4.201 self-validation discipline applied to evidence tooling). Every claim of "works" cites the analyzer's verdict + the raw evidence, so an independent re-run can reproduce the verdict.

(E) FOUR-LAYER BINDING WITH §11.4.108. At each of the four verification layers (SOURCE / ARTIFACT / RUNTIME-ON-CLEAN-TARGET / USER-VISIBLE), the layer's PASS cites MACHINE-CREATED evidence PRODUCED AT THAT LAYER (source: static-analysis JSON + review-artifact; artifact: build-log + fingerprint-checksum + byte-check of the change's landed bytes; runtime: on-target verdict + captured runtime signature + telemetry; user-visible: OCR/vision oracle output or sink-side probe report per §11.4.13/§11.4.69/§11.4.107). A PASS at a higher layer that cites evidence from a lower layer alone is a §11.4.226 wrong-layer violation — the machine-evidence chain runs THROUGH the layers, never SKIPS.

(F) STANDING DEFAULT (§11.4.126/§11.4.198). Machine-evidence-at-every-gate is the STANDING DEFAULT posture engaged from the first prompt of the session; every subagent dispatch, every review, every test run, every gate emits its evidence artifact into the project's evidence store (§11.4.207 content-addressed) so the full audit trail is machine-searchable at any time by any operator / subagent / reviewer. Prediction, guessing, and assumption about success are forbidden at the reporting layer as well as the gate layer — a status report that predicts "should pass" without citing a run's evidence is a §11.4/§11.4.1 bluff at the reporting layer.

Honest boundary (§11.4.6). §11.4.262 mandates every gate's PASS is BACKED by machine-created evidence + the analyzer is self-validated; it does NOT claim machine evidence proves human satisfaction (§11.4.185 remains, with its own machine-evidence requirement per (C)); it does NOT license shipping when the machine evidence disagrees with reality (a bug the automation missed is a §11.4.238 escape triggering a coverage audit + a vocabulary extension per §11.4.261, not a "the machine said green so we're good" defense). "Machine-created" is bounded to the artifact — the DECISION about whether a gate passed remains policy the humans authored (§11.4.6 no-guessing extends to policy calibration, not to the gate's mechanical execution).

Classification: universal (§11.4.17). Composes §11.4/§11.4.1 (anti-bluff covenant) / §11.4.5 / §11.4.6 / §11.4.13 / §11.4.15 / §11.4.20 / §11.4.27 / §11.4.50 / §11.4.66 / §11.4.69 / §11.4.85 / §11.4.98 / §11.4.107(10)(11)(12) / §11.4.108 / §11.4.110 / §11.4.115(F) / §11.4.116 / §11.4.117 / §11.4.126 / §11.4.128 / §11.4.135 / §11.4.169 / §11.4.185 / §11.4.198 / §11.4.201 / §11.4.207 / §11.4.226 / §11.4.236 / §11.4.238 / §11.4.259 / §11.4.260 / §11.4.261. Propagation gate `CM-COVENANT-114-262-PROPAGATION` (literal `11.4.262`) + recommended gates `CM-MACHINE-EVIDENCE-AT-EVERY-GATE` + `CM-EVIDENCE-ANALYZER-SELF-VALIDATED` + `CM-NO-NARRATIVE-ONLY-PASS` + `CM-EVIDENCE-LAYER-MATCH-108` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.262. Non-compliance is a release blocker. No escape hatch — no `--narrative-pass-OK`, `--operator-eyeball`, `--should-pass-suffices`, `--skip-evidence-analyzer-validation`, `--evidence-optional`, `--pass-without-machine-artifact`, `--predict-success` flag.

§ . . — Tamper-evident evidence chain + periodic anchor record: deletion, reordering, and tail-truncation of an accepted evidence record are DETECTED, not merely content-mutation (spec-derived, speckit - anti-slop-enforcement, - -)

Compact summary: every accepted evidence record (the §11.4.115(F) machine-written verdict store, the §11.4.116 event stream, or an equivalent evidence-bearing ledger a consuming project maintains) MUST be tamper-evident against THREE distinct attacks, not only the content-mutation attack §11.4.115(F)/§11.4.226 already cover: (1) deleting an entry, (2) reordering two entries, and (3) truncating the tail — and the mechanism that detects each MUST be named, because a forward-hash-chain alone detects (1) and (2) only when the chain is left internally inconsistent, and does NOT detect an attacker who deletes an entry and then RECOMPUTES the chain forward from that point (spec.md:132, FR-029's own measured boundary: *"an actor who deletes an entry and then recomputes the whole chain is NOT detected by the chain alone (0.43 s for 100k entries); that case is detected only against an anchor"*) — closing that gap requires a SEPARATE periodic anchor record the producer can append to but never rewrite (spec.md:137–139, FR-037–

039). **The mandate (ALL hold): (A) EVERY STATE-CHANGE ENTRY WITHIN THE DECLARED COVERED-CALL SET, NO CITED-READ VOLUME CREEP.** Scope (§11.4.35): this clause binds the enforcement system's own tool-call ledger under §002-anti-slop-enforcement (spec.md:129, FR-026 is written against THAT ledger, not every write/exec/deploy on the host) — the consuming project declares its own concrete COVERED-CALL SET (which state-changing operations this evidence chain tracks) as DATA per §11.4.35; §11.4.268 does NOT itself mandate a fleet-wide "every write/exec/deploy anywhere produces a chain entry" obligation, only that within the declared set, coverage is complete and cited-vs-uncited is honoured. A state-changing call WITHIN the declared covered-call set (write, exec, deploy — spec.md:129, FR-026) MUST produce a matching chain entry naming it. An UNCITED read of the store — a read whose result backs no claim — MUST NOT be required to produce an entry (spec.md:130, FR-027: *"this guards against both volume creep and a gate that refuses correct behaviour"*); a read result CITED in support of a claim MUST have a chain entry (spec.md:131, FR-028). The distinction is CITED-vs-UNCITED, never present-vs-absent-in-storage. **(B) CHAIN VERIFICATION DETECTS DELETION + REORDER WITHOUT RECONSTRUCTION.** Deleting one entry, or reordering two entries, WITHOUT reconstructing the chain that follows, MUST be reported by the integrity check (spec.md:132, FR-029) — mutation-detection of a single entry's content alone does NOT satisfy this; the check walks the FULL forward chain, recomputing every link, not spot-checking. This clause's own honest limit is stated in (C): a reconstructed (recomputed) chain defeats (B) alone. **(C) THE ANCHOR IS THE ONLY MECHANISM THAT CATCHES A RECOMPUTED-FORWARD DELETION OR A TAIL TRUNCATION.** A periodic ANCHOR entry — chain head digest AND entry count, recorded to a location the producer can APPEND to but NOT REWRITE (spec.md:137, FR-037: *"the count is what makes wholesale store deletion a detected absence rather than silence"*) — MUST be written on a declared interval. The anchor's STRENGTH is recorded honestly as `mechanism` (cryptographically or append-only enforced — a git commit history, a write-once object store, or the consuming project's declared equivalent per §11.4.35) or `policy` (procedurally enforced only); `mechanism` MUST NOT be claimed without evidence that rewriting is mechanically prevented, and absent that evidence the recorded strength is `policy` (spec.md:138, FR-038). Verification against the anchor detects a tail truncation that chain-alone verification PASSES cleanly on (spec.md:139, FR-039: *"the two results together are the evidence that the anchor, not the chain, carries this property"*) — so "the chain verified clean" is NEVER sufficient on its own; a verdict

of tamper-free MUST name BOTH the chain-walk result AND the anchor cross-check result. **(D) VERIFICATION THAT CANNOT COMPLETE REFUSES — IT NEVER REPORTS THE CHAIN INTACT.** A chain truncated mid-verification, or an anchor location unreachable at verification time, yields REFUSAL (an UNVERIFIED status, per §11.4.201's conservative-safe-default-on-an-unresolvable-signal), never a "clean pass by default" (spec.md:133, FR-030). "Could not verify" and "verified clean" are never the same reported state. **Honest boundary (§11.4.6).** §11.4.268 detects deletion, reordering, and tail-truncation of ALREADY-RECORDED entries. It does NOT prove the first entry ever written was itself truthful — the entry's CONTENT is only as trustworthy as the harness that wrote it (§11.4.115(F) machine-written, §11.4.240 producer≠verifier for who may write it), and it does NOT make the underlying evidence CORRECT, only tamper-EVIDENT. Where a project already operates a content-addressed, atomically-committed store for a related purpose (the §11.4.207 continuum resume engine), §11.4.268 REUSES that mechanism's chaining primitive rather than inventing a second one — a consuming project MUST NOT stand up a parallel, divergent chain implementation when one already satisfies this anchor's properties (§11.4.227 extend-don't-duplicate applied to mechanism reuse, not only to rule text). **Classification: universal (§11.4.17)** — a platform-neutral tamper-evidence discipline reusable by any project maintaining an evidence/verdict ledger; the consuming project supplies its concrete chain implementation, anchor location, anchor-interval, AND covered-call set (clause A — which state-changing operations this project's evidence chain tracks) as DATA per §11.4.35. Composes §11.4.5 / §11.4.6 / §11.4.69 / §11.4.115(F) (the verdict entries THIS anchor chains) / §11.4.116 (the event stream this anchor may chain) / §11.4.201 (verification-cannot-complete → refuse, the conservative-safe default) / §11.4.205 (durable/atomic/ append-only construction discipline) / §11.4.207 (reuse its content-addressed store where one already exists) / §11.4.226 (evidence-class-at-closure — a chain entry's own class must match the defect layer it backs) / §11.4.262 (machine-created evidence at every gate — the chain entries this anchor makes tamper-evident ARE the machine-created evidence artefacts §11.4.262 requires; §11.4.268 is the integrity property that keeps those artefacts trustworthy AFTER the moment of PASS, not merely present at it) / §9.2 (a genuine tamper finding is itself a destructive-op-class event requiring the §9.2 backup-and-investigate protocol) / §1.1. Propagation gate `CM-COVENANT-114-268-PROPAGATION` (literal `11.4.268` present as a block-start, exactly-once per governance file, lockstep content-hash equality across the mirror set per §11.4.227(B)) + recommended mechanism gates `CM-EVIDENCE-CHAIN-`

`DELETE-REORDER-DETECTED` (chain-walk verification detects a deleted or reordered entry left internally inconsistent) + `CM-EVIDENCE-CHAIN-ANCHOR-CATCHES-RECOMPUTE-AND-TRUNCATION` (an anchor record exists on the declared interval, carries head-digest + entry-count + an honest `mechanism` / `policy` strength, and anchor-cross-check verification detects a recomputed-forward deletion and a tail truncation that chain-alone verification misses) + `CM-EVIDENCE-CHAIN-INCOMPLETE-VERIFICATION-REFUSES` (a chain or anchor that cannot be reached/completed at verification time yields REFUSAL, never a default clean pass) + paired §1.1 mutations (delete a middle entry and recompute the chain forward from that point using only internal chain data → chain-alone verification MUST still PASS [proving the gap the anchor exists to close] while anchor cross-check MUST FAIL; truncate the tail and verify chain-alone → MUST PASS while anchor cross-check MUST FAIL; make the anchor location unreachable → verification MUST report UNVERIFIED, never PASS; golden-FALSE per §11.4.201(1): an untampered chain with a fresh, reachable, correctly-computed anchor MUST NOT fire any of the three gates). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.268 (pending landing). Non-compliance is a release blocker regardless of context. No escape hatch — no `--content-mutation-detection-suffices`, `--skip-anchor-record`, `--claim-mechanism-strength-unverified`, `--chain-truncation-passes-by-default`, `--reconstructed-chain-is-clean` flag.

---

§ . . — Critic/consensus advisory-only ban AT THE EVIDENCE-ACCEPTANCE SEAM: an ungoverned confidence/consensus signal MAY inform but MUST NEVER substitute for a receipt or adjudicate producer-verifier disagreement; role recorded `advisory`, never entered into an evidence chain — the CONSTITUTION'S MANDATORY INDEPENDENT-REVIEW-VERDICT FAMILY (§ . . /. /. /. /. /. IS UNWEAKENED (spec-derived, speckit - anti-slop-enforcement, - -)

Compact summary: at the evidence-acceptance / claim-adjudication seam — where a producer's claim of DONE/PASS is accepted or refused, and where a producer-verifier DISAGREEMENT is adjudicated — an ungoverned second opinion (a confidence score, an LLM-judge output, or a consensus aggregation across multiple such opinions, none of them produced under this constitution's own independent-review discipline) is a real, adoptable capability, but it MUST be admitted in an ADVISORY-ONLY role at THAT seam, never as the mechanism that decides accept/refuse there. This project's own operator decision on the point (spec.md:232, A-009) states the rule directly: *"Current rules adjudicate producer-verifier disagreement on the recorded receipt, never on confidence or argument quality, and explicitly bar an LLM judge from being the gate."* An ungoverned critic signal that CAN flip an accept/refuse outcome at that seam has re-created a self-certification collapse under a new name — it is a second oracle/gate sharing correlated blind spots with the producer (same model family, same training data, same failure modes as an LLM producing the work it is asked to judge), exactly the class §11.4.240/§11.4.249 already forbid a producer from occupying. **This anchor is NARROWER than "no second model may ever review a producer's work"** — it does NOT reach, weaken, override, or substitute for the constitution's SEPARATE, already-mandatory independent-review-verdict family (§11.4.125 code-review-before-build, §11.4.134 iterate-to-GO with rock-solid evidence, §11.4.142 universal every-change review, §11.4.165 independent verification agent, §11.4.209 Opus-`xhigh` review substrate, §11.4.237 translation context-and-spirit review, §11.4.256 independent per-language translation review). Those seven anchors mandate a STRUCTURALLY-SEPARATED verifier's verdict (§11.4.240 producer≠verifier, §11.4.249 oracle/gate roles) as a GENUINE receipt-generating gate on the work itself — a NO-GO from any of them BLOCKS the build (§11.4.125)

and RE-ARMS the iterate-to-GO loop (§11.4.134) — and every one of the seven remains fully in force, unmodified, unrelaxed. Clause (F) below states this unweakened-list explicitly and mechanically. **The mandate (ALL hold): (A) TWO CLOSED TERM DEFINITIONS, EXCLUDING THE MANDATORY REVIEW FAMILY.** `critic` = a second (or Nth) model instance, confidence score, or LLM-judge that offers an OPINION on a producer's claim, diff, or artifact WITHOUT itself constituting a structurally-separated, receipt-generating review conducted under §11.4.125 / §11.4.134 / §11.4.142 / §11.4.165 / §11.4.209 / §11.4.237 / §11.4.256 — i.e. an ungoverned, ad-hoc, or supplementary second opinion, not the mandatory independent-reviewer verdict those anchors already require and gate on. `consensus` = a mechanism aggregating multiple such UNGOVERNED opinions (voting, averaging, an LLM-judge score) into one combined signal. Both are ADOPTABLE capability categories a project may wire in — §11.4.269 does not forbid their existence, only their misuse as a gate AT THE EVIDENCE-ACCEPTANCE / CLAIM-ADJUDICATION SEAM. A verdict produced by the constitution's own mandatory review family is NOT a "critic" for the purposes of this anchor — it is the receipt-generating verifier role §11.4.240/§11.4.249 already govern, and it continues to gate exactly as those seven anchors require. **(B) A CLAIM WHOSE ONLY SUPPORT IS AN UNGOVERNED CRITIC/CONSENSUS SIGNAL IS REFUSED.** Per spec.md:163 (FR-023): *"Given a claim whose only support is a critic consensus or confidence score, the seam MUST refuse it"* — naming the missing receipt. A recorded receipt — a machine-written §11.4.115(F) verdict, a captured evidence-chain entry per §11.4.268, OR a mandatory independent review's own recorded GO/NO-GO under §11.4.125/§11.4.134/ §11.4.142/ §11.4.165/§11.4.209/§11.4.237/§11.4.256 — remains the ONLY basis on which producer-verifier disagreement is adjudicated; confidence or argument quality UNBACKED by such a receipt, however produced, is never a substitute. **(C) DETERMINISTIC WITH AND WITHOUT THE UNGOVERNED CRITIC.** Per spec.md:164 (FR-024): the SAME claim, evaluated once with an UNGOVERNED critic step present and once with it absent, MUST produce the IDENTICAL accept/ refuse outcome. A pipeline in which disabling that ungoverned critic changes the verdict is, by definition, using it as a gate — the deterministic-outcome test is the mechanical proof that (B) is genuinely honoured, not merely stated. (This clause does NOT apply to a mandatory review under the seven-anchor family in clause (A) — skipping a REQUIRED §11.4.125/§11.4.134/§11.4.142 review is itself a violation of those anchors, not a deterministic-outcome test to pass; the test in this clause is scoped to the OPTIONAL, ungoverned critic only.) **(D) UNGOVERNED CRITIC**

OUTPUT IS RECORDED `advisory` IN A SEPARATE STORE, NEVER ENTERED INTO AN EVIDENCE CHAIN. Per spec.md:165 (FR-025): ungoverned critic output MUST be recorded with a role of `advisory` and MUST NOT appear in any §11.4.268/§11.4.115(F) evidence chain — it is recorded in a separate advisory store, never intermixed with the accept/refuse evidence chain itself. A mandatory review's OWN recorded GO/NO-GO under the seven-anchor family continues to be entered into the evidence/verdict record as its own anchors require (§11.4.125's review-completed marker, §11.4.134's clean-GO record) — it is not retroactively downgraded to `advisory` by this clause. **(E) WHY THE UNGOVERNED-CRITIC COLLAPSE MATTERS.** Composes §11.4.240(A)/(C) and §11.4.249 — an ungoverned critic wired into the evidence-acceptance gate collapses ORACLE and GATE into a single mechanism that is NOT independent of the producer's own failure surface (correlated model errors, shared blind spots), which is precisely the class of collapse §11.4.240 names as maximum-severity. Advisory-only admission avoids re-creating that collapse under the label "review" while retaining the genuine usefulness of an ungoverned critic — surfacing likely issues earlier, feeding §11.4.102 systematic debugging, prioritising a human reviewer's attention — none of which requires it to hold gating authority. The mandatory seven-anchor review family avoids the SAME collapse by a DIFFERENT mechanism entirely — structural separation of the reviewer per §11.4.240, not advisory-demotion of its verdict — which is exactly why that family's verdicts remain full receipts and this anchor never touches them. **(F) THIS BAN IS SCOPED TO THE EVIDENCE-ACCEPTANCE / CLAIM-ADJUDICATION SEAM — IT DOES NOT WEAKEN, OVERRIDE, OR SUBSTITUTE FOR THE MANDATORY INDEPENDENT-REVIEW-VERDICT FAMILY.** §11.4.125 (code-review-agent gate before pre-build + main build), §11.4.134 (iterate-to-GO with rock-solid evidence), §11.4.142 (universal code-review — every change reviewed, always), §11.4.165 (independent verification agent), §11.4.209 (code-review MUST run on the Opus model at `xhigh` effort), §11.4.237 (context-and-spirit translation review), and §11.4.256 (independent per-language translation review) each mandate a STRUCTURALLY-SEPARATED verifier (§11.4.240 producer≠verifier) whose VERDICT genuinely gates, and NONE of these seven anchors is relaxed, narrowed, made advisory, or made optional by §11.4.269. What §11.4.269 forbids is narrower and different: (i) treating an UNGOVERNED second opinion / confidence score — one NOT produced under any of those seven anchors' own independence-plus-evidence discipline — as a substitute receipt (clause B); and (ii) letting such an ungoverned signal, rather than a recorded receipt (including a receipt produced by the seven-

anchor family), adjudicate a producer-verifier DISAGREEMENT (the A-009 scope, spec.md:232). A project MUST NOT cite §11.4.269 to justify skipping, weakening, downgrading-to-advisory, or making optional any of the seven listed mandatory reviews — doing so is itself a §11.4.269(F) violation, exactly the mis-scoping this clause exists to foreclose. **Honest boundary (§11.4.6).** §11.4.269 does not forbid using an ungoverned critic/consensus signal to prioritise human review, to surface candidate defects for §11.4.102 investigation, or as ONE input among many an independent human/verifier weighs when authoring (not executing) a gate. It forbids only its use as an automated GATING mechanism substituting for a real, receipt-backed oracle/gate/verifier chain. It does not claim critics are useless — only that their output is advisory context, never a verdict, UNLESS that output is itself produced under the mandatory independent-review-verdict family named in clause (F), in which case it is a receipt and this anchor does not touch it. **Classification: universal (§11.4.17)** — a platform-neutral discipline reusable by any project that adopts, or is asked to adopt, a second-model/critic/consensus review capability OUTSIDE its own mandatory independent-review-verdict family; the consuming project supplies its concrete critic implementation and advisory-record location as DATA per §11.4.35, and its own set of already-mandatory review-verdict anchors (analogous to this project's §11.4.125/ .134/.142/.165/.209/.237/.256) as the fixed, unweakened-by-this-anchor list. Composes §11.4.6 / §11.4.69 / §11.4.115(F) / §11.4.125 / §11.4.134 / §11.4.142 / §11.4.165 / §11.4.209 / §11.4.237 / §11.4.256 (the seven mandatory review-verdict anchors clause (F) leaves unweakened) / §11.4.240 (the separation principle this anchor is a specific instantiation of) / §11.4.249 (the four-role architecture that names why an ungoverned critic-as-gate is a collapse) / §11.4.268 (the evidence chain the ungoverned critic's output MUST NOT enter) / §11.4.102 (the legitimate advisory use — feeding investigation) / §1.1. Propagation gate `CM-COVENANT-114-269-PROPAGATION` (literal `11.4.269` present as a block-start, exactly-once per governance file, lockstep content-hash equality across the mirror set per §11.4.227(B)) + recommended mechanism gate `CM-CRITIC-CONSENSUS-ADVISORY-ONLY` (a claim supported only by an ungoverned critic/consensus/confidence signal, with no receipt, is refused; the accept/refuse outcome for a claim is identical whether an ungoverned critic step ran or was skipped; every stored ungoverned-critic output carries role `advisory` and appears in no evidence chain; a mandatory review under §11.4.125/.134/.142/.165/.209/.237/.256 is NEVER downgraded to `advisory` or excluded from gating by this gate) + paired §1.1 mutation (wire an ungoverned critic's approve/reject value directly into the pass/fail decision → the gate MUST FAIL; store an ungoverned

critic output with no role field, or role `verdict`, or entered into the evidence chain → the gate MUST FAIL; downgrade a §11.4.125/.134/.142/.165/.209/.237/.256 review's own GO/NO-GO record to `advisory`, or exclude it from gating, citing §11.4.269 as the reason → the gate MUST FAIL; golden-FALSE per §11.4.201(1): (i) an ungoverned critic present, its output stored with role `advisory`, absent from the evidence chain, and the deterministic outcome unchanged whether it ran or not → the gate MUST NOT fire; (ii) a §11.4.125 code-review's own NO-GO record blocking the build exactly as §11.4.125 requires → the gate MUST NOT fire). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.269 (pending landing). Non-compliance is a release blocker regardless of context. No escape hatch — no `--critic-may-gate`, `--consensus-score-is-a-receipt`, `--llm-judge-is-the-gate`, `--critic-output-in-evidence-chain-OK`, `--outcome-may-differ-with-critic`, `--269-overrides-mandatory-review`, `--downgrade-mandatory-review-to-advisory` flag.

---

§ . . — Dependency-existence-verdict register: every proposed dependency carries a closed-set existence verdict {VERIFIED, AMBIGUOUS, UNVERIFIED} with citable evidence; adoption is gated behind resolution and behind the wiring-sweep result (spec-derived, speckit -anti-slop-enforcement, - -)

Compact summary: a project that adopts an external (or internal, cross-module) dependency to close a coverage gap MUST first record, for that dependency, a closed-set EXISTENCE verdict — VERIFIED, AMBIGUOUS, or UNVERIFIED — backed by a citable artifact or an explicit statement that none was found (spec.md:114, FR-013). Existence verification is NECESSARY but NOT SUFFICIENT for adoption (spec.md:226, A-012: *"Existing is not the same as working ... Several verified entries are at initial release versions, one declares a repository that returns 404, and one's 'tamper-evident' claim is on inspection only 'append-only' — which detects nothing if the appender is the party being checked"*) — a VERIFIED existence verdict is the FLOOR a dependency clears before it may even be considered adopted, never a claim that it is fit for the purpose it is being

adopted for. **The mandate (ALL hold): (A) THE REGISTER, ONE ROW PER CLAIMED DEPENDENCY.** Every dependency a project proposes to adopt gets exactly one register row, keyed by a stable identifier (name + version + source per §11.4.111 resolve-by-stable-name). The row carries the verdict + its evidence field (spec.md:114, FR-013: *"MUST carry a verdict of VERIFIED, AMBIGUOUS or UNVERIFIED with a citable artifact or an explicit statement that none was found"*). **(B) VERDICT SEMANTICS, EVIDENCE-BASED, NEVER PROMOTED BY OMISSION.** UNVERIFIED is the honest default state — a claim about a dependency's existence with no independent confirmation attempted MUST NOT be silently treated as VERIFIED. AMBIGUOUS is assigned when evidence conflicts or a name collision exists — spec.md:116 (FR-015): *"a dependency name that collides with an unrelated project" gets AMBIGUOUS, never VERIFIED* — the specific conflicting evidence is recorded, not merely a vague "unsure". VERIFIED requires an independently-reproducible check: a real resolve + checksum/hash match, a real successful call or import, or an equivalent control-needle-proven presence per §11.4.201(7)(b). **(C) ADOPTION REFUSED WHILE THE VERDICT IS NON-VERIFIED.** Per spec.md:115 (FR-014): *"Given a dependency whose verdict is UNVERIFIED or AMBIGUOUS, adoption MUST be refused."* Adoption is admitted ONLY after the row resolves to VERIFIED (or the AMBIGUOUS conflict is investigated and the row is re-verdicted). **(D) ADOPTION IS GATED BEHIND MAPPING TO A RECORDED UNCOVERED GAP + BEHIND THE WIRING-SWEEP RESULT.** Per spec.md:170 (FR-020): adoption is refused unless the dependency maps to one of the project's own recorded uncovered capability gaps (adopting a dependency for a capability the project already covers is itself refused, naming the covering rule — the §11.4.227 duplicate-coverage discipline applied to dependency adoption). Per spec.md:171 (FR-021): *"the adoption step MUST NOT be accepted before the wiring step reports its sweep result"* — a project does not decide "yes, adopt" before it has run the sweep that would reveal whether the dependency's claimed capability is genuinely reachable and wired (§11.4.124/ §11.4.196(F) investigate-before-adopt, the mirror-image of investigate-before-remove applied to the opposite direction). **(E) ZERO ADOPTED WITHOUT VERIFIED — MONOTONIC.** Per spec.md:217 (SC-005): the count of external dependencies adopted without a VERIFIED existence verdict is ZERO, checked at the SAME seam that already refuses unwired-gate names per §11.4.227(A) (the named-gate ledger), generalised here to dependency-adoption claims. **Honest boundary (§11.4.6).** §11.4.270's register proves EXISTENCE/RESOLVABILITY was checked and recorded — it does NOT prove the dependency BEHAVES

correctly for the purpose it is adopted for (that stays ordinary integration/contract testing per §11.4.244), and it does NOT prove the dependency is SECURE or well-maintained (that is §11.4.246's supply-chain territory). A-012's own finding — "existing is not the same as working" — is why this anchor is deliberately narrow: it closes the EXISTENCE-CLAIM bluff, not the fitness-for-purpose question, which stays governed by the constitution's ordinary testing/review family. **Classification: universal (§11.4.17)** — a platform-neutral existence-verification discipline reusable by any project that adopts external or cross-module dependencies to close a capability gap; the consuming project supplies its register location, its recorded uncovered-gap set, and its wiring-sweep mechanism as DATA per §11.4.35. Composes §11.4.6 / §11.4.69 / §11.4.111 (stable-name keying) / §11.4.124 (investigate-before-adopt, the direction-mirror of investigate-before-remove) / §11.4.196(F) (configured ≠ wired, the same principle applied to a dependency's claimed capability) / §11.4.201(7)(b) (control-needle-proven presence checks) / §11.4.227(A) (named-gate ledger — this anchor generalises its zero-unwired-debt discipline to dependency claims) / §11.4.244 (fitness-for-purpose stays contract-testing territory, distinct from existence) / §11.4.246 (security/supply-chain, also distinct from existence) / §11.4.74 (submodule-catalogue-first discovery + extend-don't-reimplement — the register's VERIFIED-tier check IS the mandated catalogue-first discovery step, applied to any dependency, not only own-org submodules) / §11.4.31 (submodule-dependency-manifest — an owned-org dependency's `helix-deps.yaml` is itself citable evidence toward a VERIFIED verdict) / §11.4.150 (mandatory deep multi-angle web research before declaring a capability covered — the research pass that PRODUCES a register row's citable evidence, or its explicit none-found statement) / §1.1. Propagation gate `CM-COVENANT-114-270-PROPAGATION` (literal `11.4.270` present as a block-start, exactly-once per governance file, lockstep content-hash equality across the mirror set per §11.4.227(B)) + recommended mechanism gate `CM-DEPENDENCY-EXISTENCE-VERDICT-REGISTER` (every claimed dependency has a register row carrying a closed-set verdict + evidence field; a name collision resolves AMBIGUOUS never VERIFIED; a shipped change relying on an UNVERIFIED or AMBIGUOUS row is refused; adoption refused when the dependency maps to an already-covered capability or precedes its wiring-sweep result) + paired §1.1 mutation (adopt a dependency with no register row → the gate MUST FAIL; leave a row UNVERIFIED and adopt anyway → the gate MUST FAIL; feed a name-collision fixture and assert the verdict resolves VERIFIED instead of AMBIGUOUS → the gate MUST FAIL; golden-FALSE per §11.4.201(1): a dependency VERIFIED with

real, reproducible, control-needle-proven evidence, mapped to a genuinely-uncovered gap, adopted after its wiring sweep reported a result → the gate MUST NOT fire). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.270 (pending landing). Non-compliance is a release blocker regardless of context. No escape hatch — no `--unverified-adoption-OK`, `--ambiguous-treated-as-verified`, `--skip-wiring-sweep-before-adopt`, `--adopt-already-covered-capability`, `--existence-proves-fitness` flag.

---

§ . . — Waiver mechanism: the strict FORMALIZATION of allow-with-tracked-debt (unifying § . . (D)'s recorded-deferral, § . . ( )'s explicit-operator-override, and § . . (A)'s deadline-bound quarantine into ONE roster + expiry + tracked-item schema), distinct from refusal (§ . .) and closure (§ . .) vocabulary (spec-derived, speckit -anti-slop-enforcement, - -)

Compact summary: every project needs, on rare occasion, a mechanism to ALLOW a change to proceed despite a normally-blocking missing-evidence condition — never a silent, never a permanent bypass, but a tracked, expiring, human-authorised exception. **This is NOT an absent capability being introduced here** — re-verified in this remediation pass: §11.4.234(D) already permits an explicit, documented deferral flag ("The escape hatch is a recorded deferral, not a bypass: the skipped gate remains owed and is caught at the next full run or tag gate" — Constitution.md:10999); §11.4.236(4) already requires "an explicit, logged, operator-set flag" for any override of its deploy-readiness seam ("EXPLICIT, RECORDED OVERRIDE ONLY" — Constitution.md:11023); and §11.4.248(A) already ships a deadline-bound quarantine for one specific gate class (flaky tests, moved to `tests/quarantine/` with a tracked §11.4.197 stabilisation deadline). Those three are genuine allow-with-known-debt PRECEDENTS this anchor does not discover but FORMALIZES. What §11.4.271 genuinely adds — needled: `waiv*` returns ZERO hits anywhere in `constitution/Constitution.md` outside

this draft, confirming the SCHEMA below is new even though the CATEGORY of mechanism is not — is the STRICT, DATA-BEARING, cross-gate schema those three precedents describe only informally and per-mechanism: a ROSTERED NON-PRODUCER authoriser (§11.4.234(D) and §11.4.248(A) name no authoriser; §11.4.236(4) names the operator as a role but no roster-resolvable identity schema — §11.4.271 adds the roster resolution + producer exclusion), a MANDATORY UNELAPSED EXPIRY (§11.4.234's skip flag and §11.4.236's override flag carry no expiry field at all — an indefinite skip/override is currently legal under their text), and a MANDATORY TRACKED ITEM (neither §11.4.234(D) nor §11.4.236(4) requires one; §11.4.248(A) requires it for its one gate class — §11.4.271 makes it mandatory cross-gate). §11.4.271 turns three ad-hoc, per-anchor escape hatches into ONE cross-cutting mechanism any future gate can invoke by reference instead of re-inventing its own — this project's own `scripts/lib/critical_blocker_gate.sh` (built for the §11.4.236 seam) already implements `waive` / `override` operations by name, direct evidence this schema generalises code the project has already written, not a speculative addition. **The mandate (ALL hold): (A) THE WAIVER IS THE ONE FORMALIZED SCHEMA FOR AN ALLOW-WITH-KNOWN-DEBT PATH.** A waiver EXISTS at a seam that would otherwise refuse (a missing receipt, an unresolved dependency verdict, an unmet acceptance-criteria scenario, or any other normally-blocking condition this constitution's family of gates enforces) — the SAME class of seam §11.4.234(D)'s deferral flag, §11.4.236(4)'s override flag, and §11.4.248(A)'s quarantine already gate. It is invoked explicitly, never inferred, and it never disables the gate permanently — it grants a bounded, tracked exception to ONE candidate. A gate ALREADY implementing an allow-with-known-debt path under §11.4.234/ §11.4.236/§11.4.248 is NOT required to reimplement one from scratch — it satisfies §11.4.271 by conforming its existing flag/quarantine mechanism to this clause's roster + expiry + tracked-item schema (clauses B–D below), the strict superset those three anchors' informal mechanisms currently lack. **(B) AUTHORISATION MUST RESOLVE TO A ROSTERED IDENTITY, NEVER THE PRODUCER.** Per spec.md:157 (FR-040): *"Given a waiver whose authorisation value does not resolve to an identity in the declared authoriser roster, the seam MUST refuse it"* — a free-text authorisation ("looks fine") is refused exactly as an absent one is; a waiver naming a genuinely rostered identity is accepted (the false-positive guard — a check that refuses EVERY waiver has replaced a bypass with a deadlock, not fixed anything). The authoriser roster is consumer-supplied DATA per §11.4.35, and the producer of the change under waiver MUST NOT appear as its own waiver's

authoriser (composes §11.4.240 producer≠verifier — a producer cannot waive their own gate). **(C) EXPIRY IS REQUIRED AND MUST NOT HAVE ELAPSED.** Per spec.md:158 (FR-041): a waiver with NO expiry, or whose expiry HAS elapsed, is refused; a waiver whose expiry is in the future is accepted. An absent expiry is NOT read as "valid forever" — the spec's own framing is exact: *"An expiry absent by omission is not 'valid for every candidate forever' — that is the same downgrade-by-omission the satisfaction path already refuses"* elsewhere in this family (§11.4.146(D3)/§11.4.115(F) fingerprint-absent-is-refused, the general pattern this clause instantiates for waivers). An elapsed waiver's gate reverts to blocking automatically — no grace period, no silent auto-renewal. **(D) A TRACKED ITEM NAMES THE DEBT; A WAIVER NAMING NONE IS REFUSED.** Per spec.md:159 (FR-042): *"Given a waiver, it MUST name a tracked item for the blocker that remains open, and a waiver naming none MUST be refused"* — because *"a waiver is the only ALLOW that leaves the defect unfixed, so it is the one path on which the owed work must stay traceable."* The tracked item is an ordinary §11.4.54-stable-id workable item bound to the SAME lifecycle discipline (§11.4.15/§11.4.16/§11.4.33/§11.4.197) as any other tracked work — a waiver whose tracked item never resolves is itself surfaced by the standing zero-shortcomings audit (§11.4.261) as an unresolved-waiver finding, never silently forgotten. **(E) COMPOSITION, NOT SUBSTITUTION — INCLUDING WITH THE THREE PRECEDENT MECHANISMS.** A waiver whose expiry lapses UNRESOLVED re-enters the §11.4.21 Operator-blocked class for its underlying condition (the block returns; nothing about the original blocking condition changed, only the temporary permission to pass did). A waiver's tracked item CAN close via §11.4.90 Obsolete if the underlying requirement genuinely changed. Neither §11.4.21 nor §11.4.90 substitutes for the waiver mechanism itself — a project attempting to express "allow despite missing evidence, tracked, expiring" via either of those two vocabularies is misusing them, per this anchor's own honest-boundary clause. Symmetrically, §11.4.234(D)'s deferral flag, §11.4.236(4)'s override flag, and §11.4.248(A)'s quarantine are NOT replaced by §11.4.271 — each remains the correct, narrower mechanism for its own seam class; §11.4.271 is the schema they conform TO when a project wants their allow-with-debt behaviour to carry a rostered authoriser, an expiry, and a tracked item, not a rival mechanism competing with them. **Honest boundary (§11.4.6).** §11.4.271 formalizes, it does not discover, the ALLOW-with-tracked-debt CATEGORY — §11.4.234(D), §11.4.236(4), and §11.4.248(A) already establish that category exists in this constitution; why §11.4.271 does not make waivers routine or easy is the SCHEMA's own shape (rostered-non-producer

authorisation, mandatory unelapsed expiry, mandatory tracked item) — deliberately strict, because a waiver is the ONE path this constitution's anti-bluff family permits to leave a defect genuinely unfixed while still shipping. It does not weaken any gate a waiver bypasses for OTHER candidates, and it does not authorise an indefinite or permanently-renewed waiver — an expiry that keeps getting pushed forward without the underlying debt closing is itself a §11.4.261 finding (a monotone-decreasing-findings-ledger violation, not a compliant waiver renewal pattern). **Classification: universal (§11.4.17)** — a platform-neutral, deliberately-strict allow-with-tracked-debt discipline reusable by any project whose gate family needs an occasional, bounded, evidence-honest exception; the consuming project supplies its authoriser roster, expiry mechanism, and tracked-item store as DATA per §11.4.35. Composes §11.4.6 / §11.4.15 / §11.4.16 / §11.4.21 (refusal vocabulary — distinct, cited not restated) / §11.4.33 (closure vocabulary) / §11.4.35 / §11.4.54 (stable-id tracked item) / §11.4.90 (closure vocabulary — distinct, cited not restated) / §11.4.146(D3) (the absent-field-is-refused pattern this anchor applies to expiry + authoriser + tracked-item) / §11.4.197 (the tracked item's completion obligation) / §11.4.234(D) (recorded-deferral precedent this anchor formalizes) / §11.4.236(4) (explicit-recorded-override precedent this anchor formalizes) / §11.4.248(A) (deadline-bound quarantine precedent this anchor formalizes for one gate class) / §11.4.240 (producer≠verifier — the authoriser exclusion) / §11.4.261 (an unresolved waiver is a standing-audit finding) / §1.1. Propagation gate `CM-COVENANT-114-271-PROPAGATION` (literal `11.4.271` present as a block-start, exactly-once per governance file, lockstep content-hash equality across the mirror set per §11.4.227(B)) + recommended mechanism gate `CM-WAIVER-ROSTERED-EXPIRY-TRACKED` (every active waiver resolves to a genuinely rostered, non-producer authoriser identity; a future, unelapsed expiry; and a named §11.4.54-stable-id tracked item; a waiver missing ANY ONE of the three is invalid and its waived gate reverts to blocking) + paired §1.1 mutation (submit a waiver with free-text or absent authorisation → the gate MUST FAIL; submit one with no expiry, or an elapsed expiry → the gate MUST FAIL; submit one naming no tracked item → the gate MUST FAIL; submit one where the authoriser identity equals the change's producer → the gate MUST FAIL; golden-FALSE per §11.4.201(1): a waiver naming a genuinely-rostered non-producer identity, a future expiry, and a real tracked item → the gate MUST NOT fire). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.271 (pending landing). Non-compliance is a release blocker regardless of context. No escape hatch — no `--free-text-`

`authorisation-OK`, `--waiver-without-expiry`, `--elapsed-waiver-still-valid`, `--waiver-without-tracked-item`, `--producer-may-self-waive`, `--permanent-waiver-renewal` flag.